

Learning



Aprendizaje No Supervisado: Supervisado: Descubriendo Patrones Ocultos

El aprendizaje no supervisado analiza datos sin sin etiquetas para encontrar estructuras ocultas. A ocultas. A diferencia del supervisado, busca patrones sin guía externa.

Aprendizaje Supervisado vs. No Supervisado

Supervisado

Utiliza datos etiquetados para entrenar modelos predictivos.
predictivos.

Requiere ejemplos con respuestas correctas.

Enfocado en predicción y clasificación.

No Supervisado

Trabaja con datos sin etiquetar.

Busca patrones y relaciones inherentes.

Descubre estructuras ocultas en los datos.

Tipos Principales de Aprendizaje No Supervisado

Clustering (Agrupamiento) (Agrupamiento)

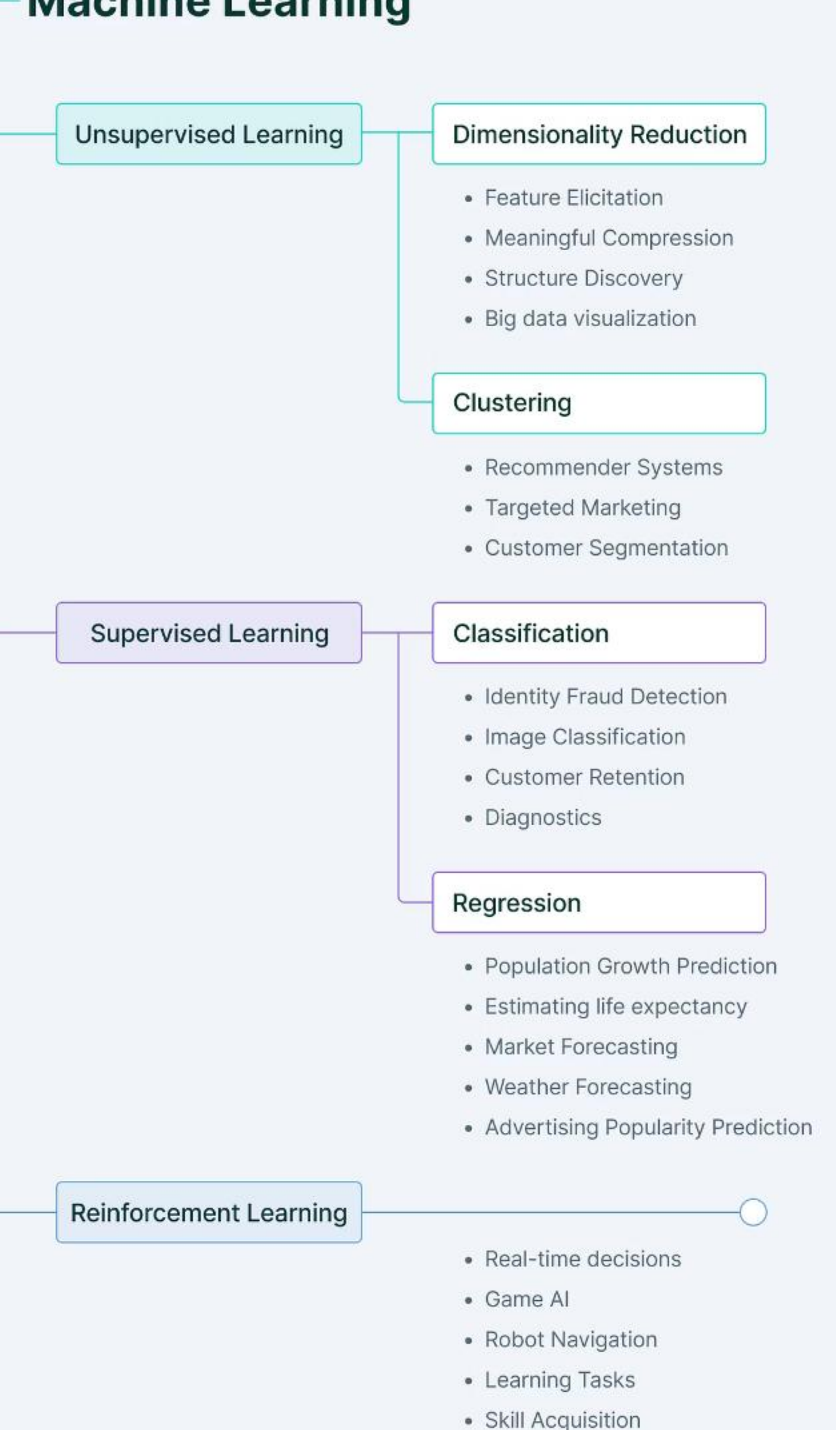
Agrupa datos similares en clusters basados en características comunes.

Reducción de Dimensionalidad

Simplifica datos manteniendo su estructura esencial.

Detección de Anomalías

Identifica patrones inusuales o atípicos en los datos.



Algoritmos de Clustering



K-means

Divide los datos en k grupos
grupos basados en la
proximidad a los centroides.
centroides.



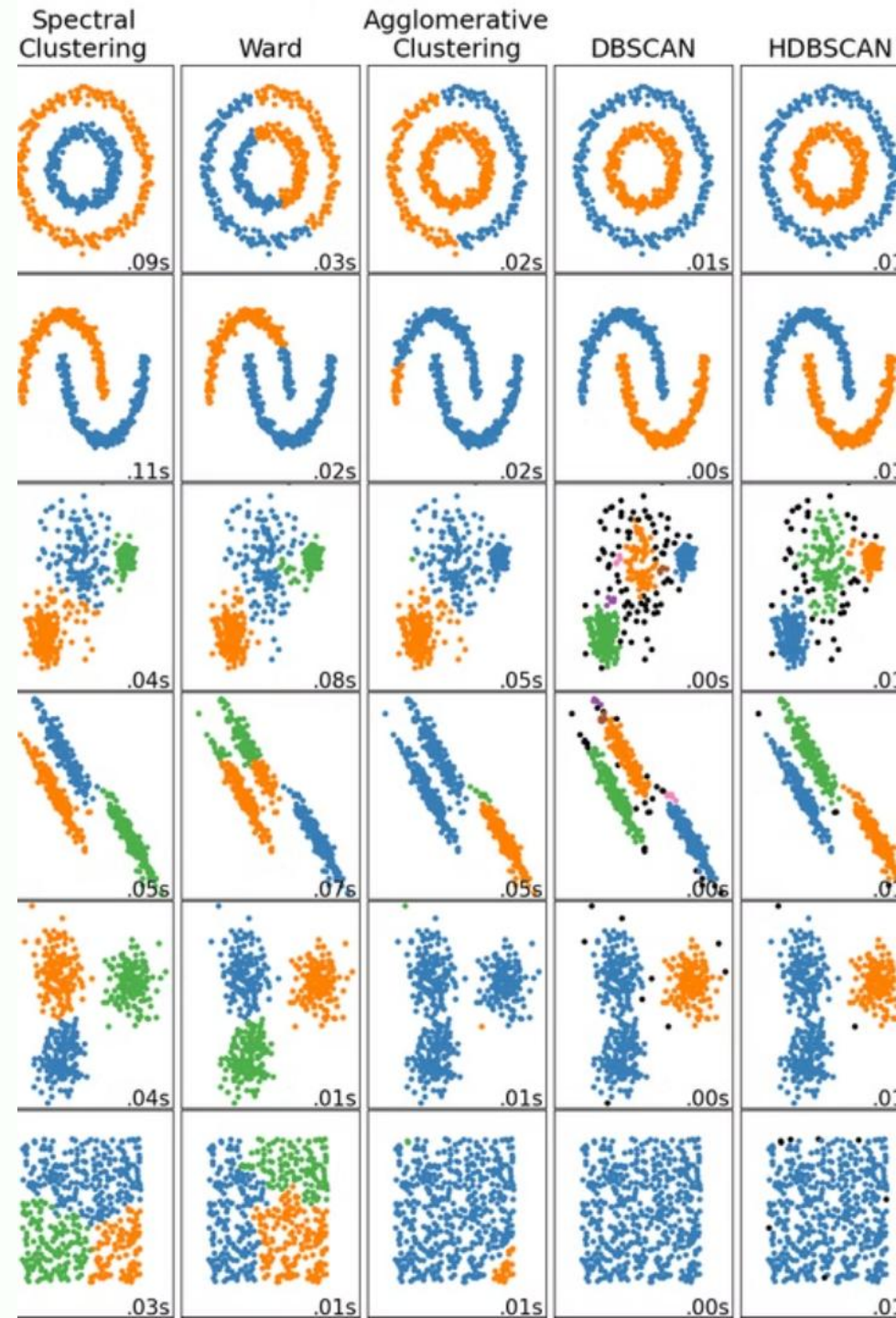
DBSCAN

Agrupar puntos cercanos según una densidad mínima, identificando ruido y
identificando ruido y outliers.



Clustering Jerárquico

Crea una jerarquía de clusters
clusters mediante fusiones o
o divisiones sucesivas.



Técnicas de Reducción de Dimensionalidad



PCA

Transforma los datos a un nuevo espacio con menos dimensiones.



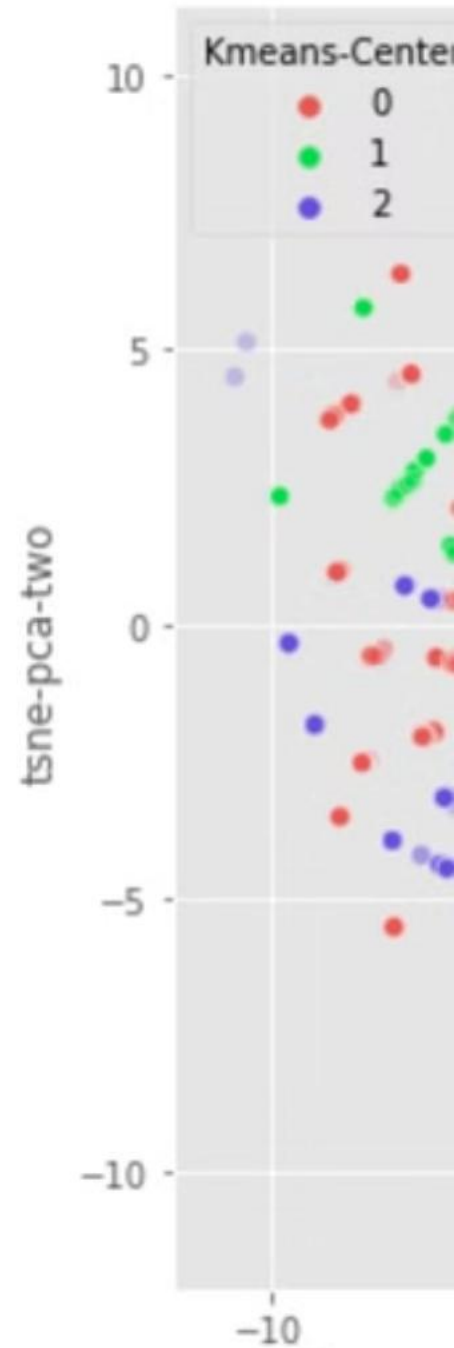
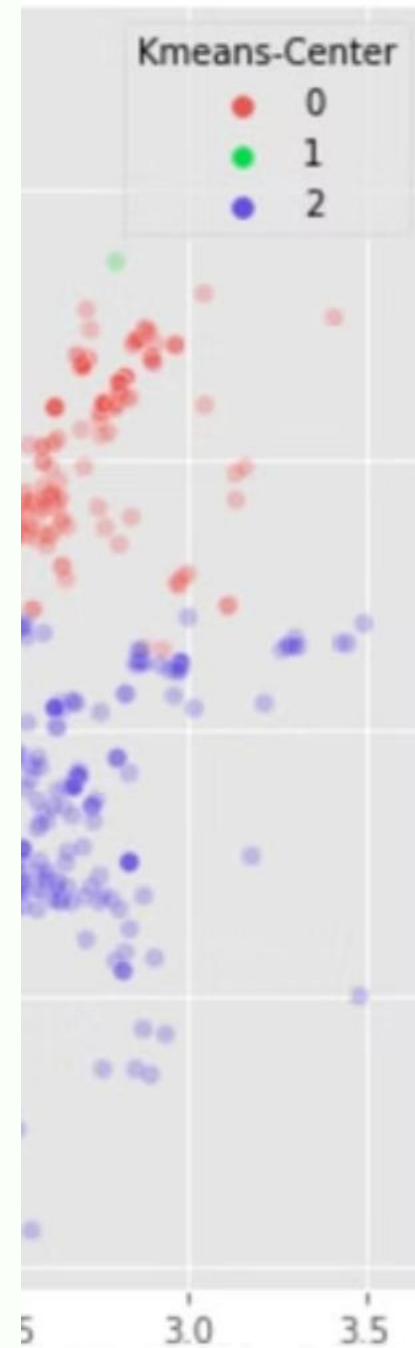
t-SNE

Visualiza datos de alta dimensión en 2D o 3D, preservando relaciones locales.



UMAP

Técnica moderna que preserva tanto la estructura local como global.



Ejercicio Interactivo: Agrupando Frutas



Por Color

Rojas (manzana, fresa), amarillas (plátano, (plátano, piña), verdes (kiwi, uva).



Por Tamaño

Pequeñas (cereza, uva), medianas (manzana, (manzana, naranja), grandes (sandía, melón). melón).

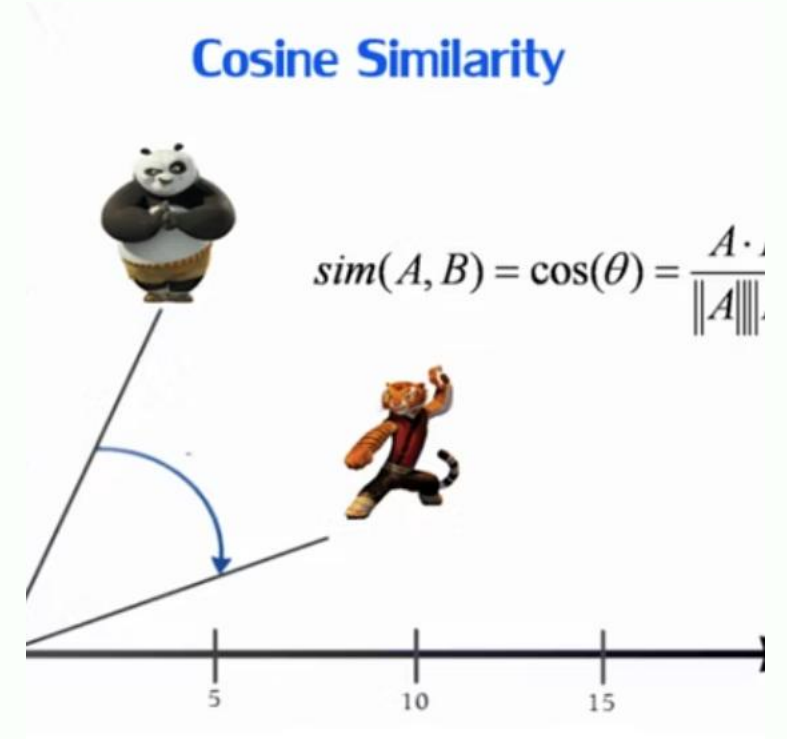
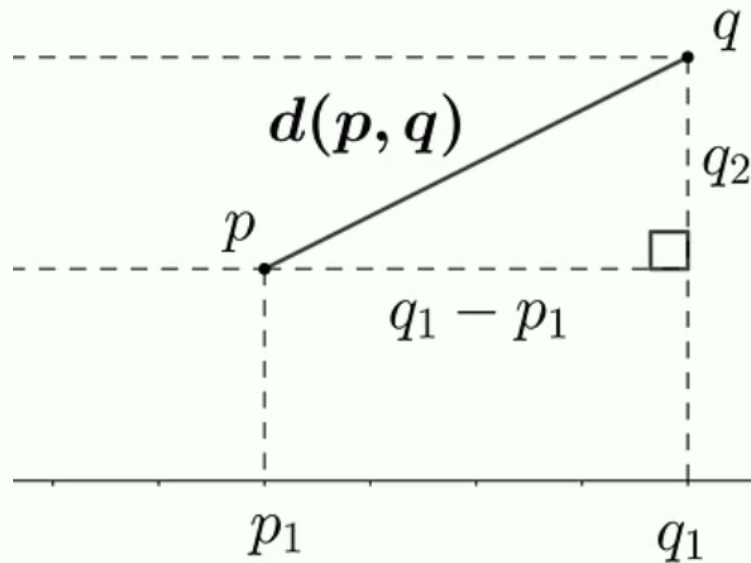


Por Forma

Redondas (manzana, naranja), alargadas alargadas (plátano, pepino).

Concepto de Distancia y Similitud

$$\|p - q\|^2 = (q_1 - p_1)^2 + (q_2 - p_2)^2$$



La distancia mide diferencias entre puntos. La similitud indica cuán parecidos son. Ambas definen cómo agrupar datos en clustering.

Tipos de Distancias



Euclidiana

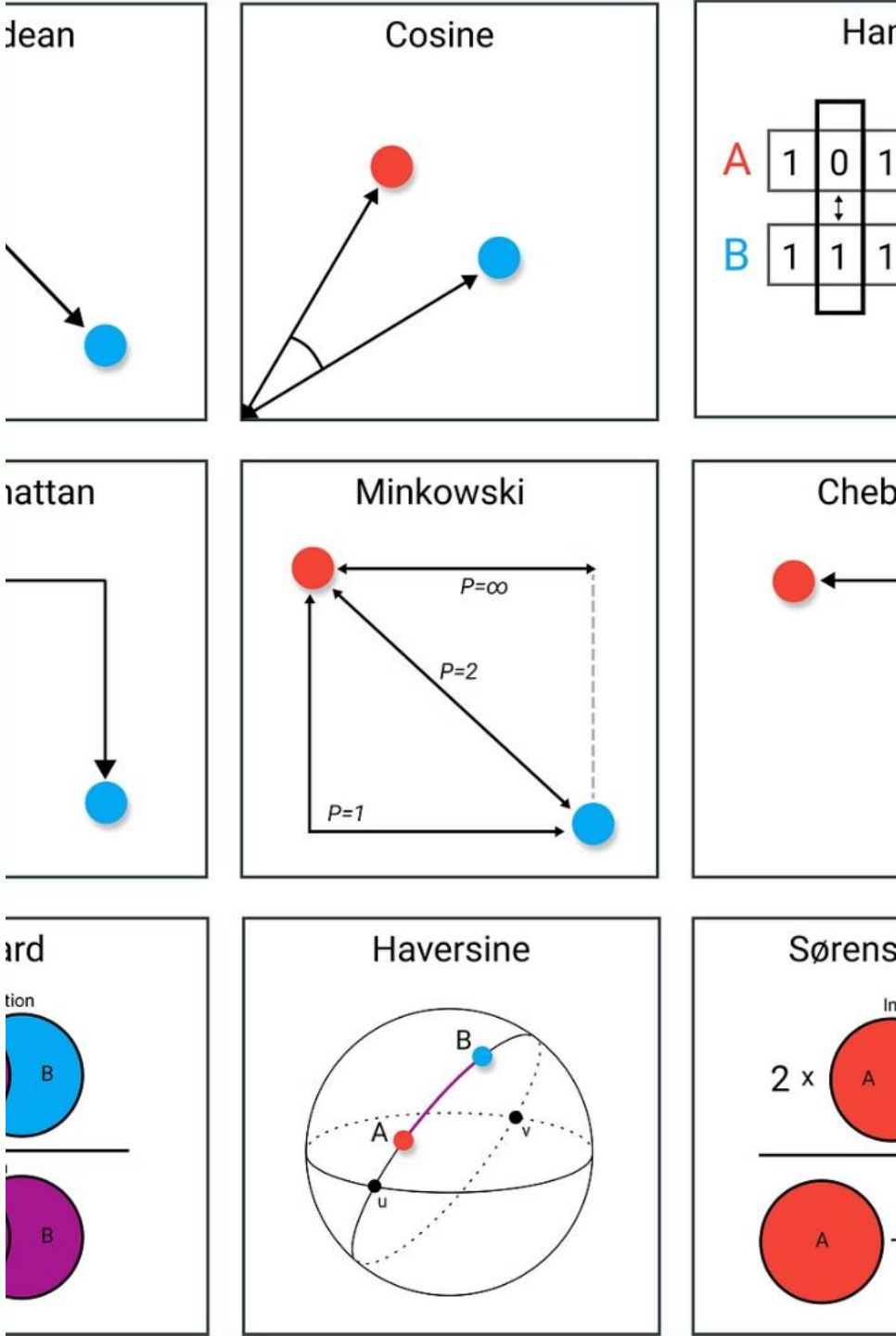
Recta en un espacio n-dimensional.

Manhattan

Camino ortogonal en cuadrícula.

Coseno

Basada en el ángulo entre vectores.



requesting does not exist
or is no longer available.

Clustering Particional: K-Means

Inicializar k centroides al azar

Se eligen puntos aleatorios como centroides iniciales.

Asignar puntos al centroide centroide más cercano

Cada dato se asigna al grupo con centroide más próximo.

Recalcular centroides

Se actualiza la posición de cada centroide según los puntos asignados.

Repetir hasta convergencia convergencia

El proceso continúa hasta que los centroides se estabilizan.

Clustering Jerárquico



Clustering Basado en Densidad:

Densidad: DBSCAN



Seleccionar un punto no visitado

Se comienza con un punto aleatorio del conjunto de datos.



Verificar densidad

Si hay suficientes puntos cercanos, forma un nuevo cluster.



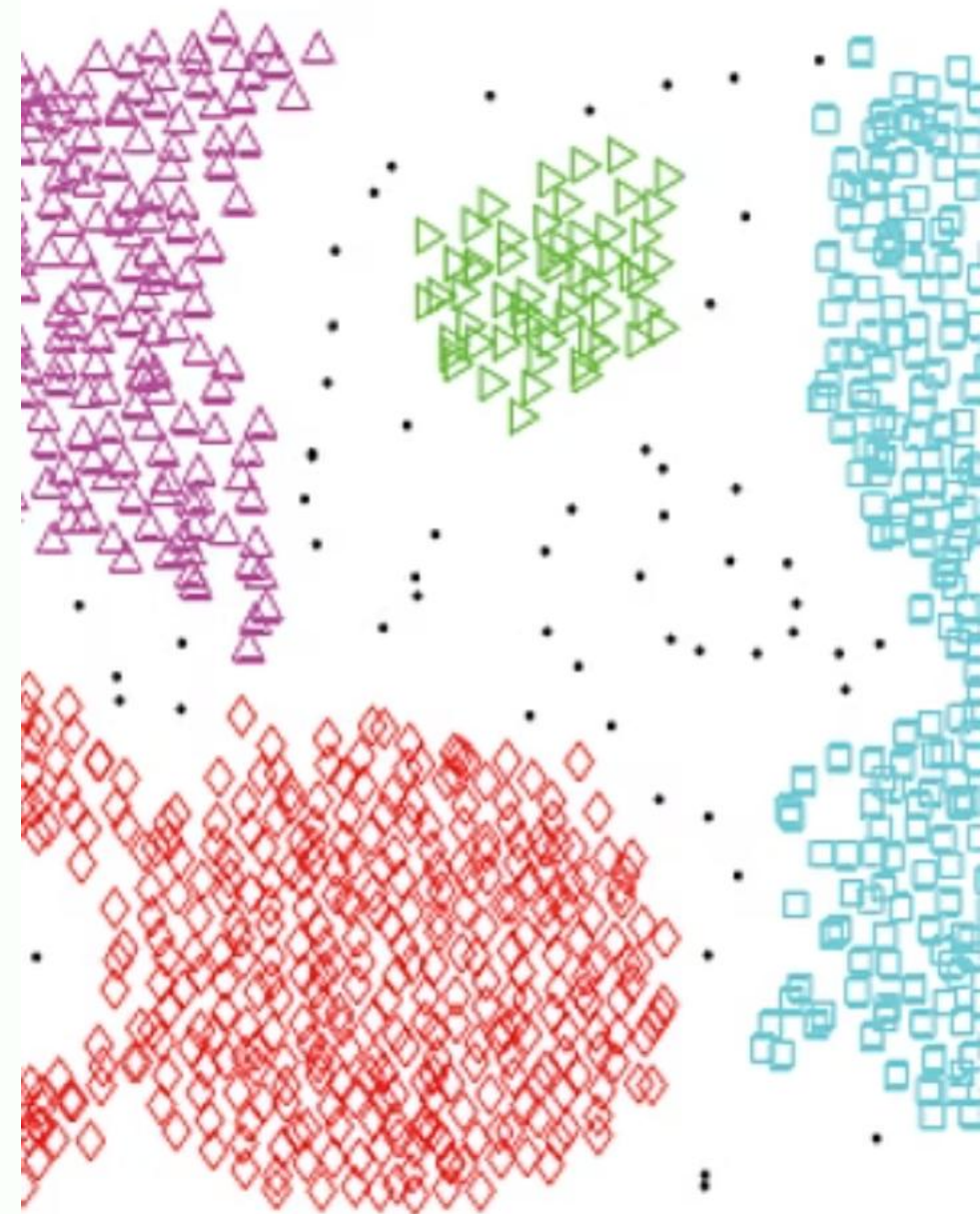
Expandir el cluster

Incluye vecinos densos y continúa el proceso recursivamente.



Marcar ruido

Los puntos que no cumplen condiciones se marcan como outliers.



(b)

Visualización de Clusters



Gráficos 2D

Scatterplots para datos con pocas características



Reducción previa

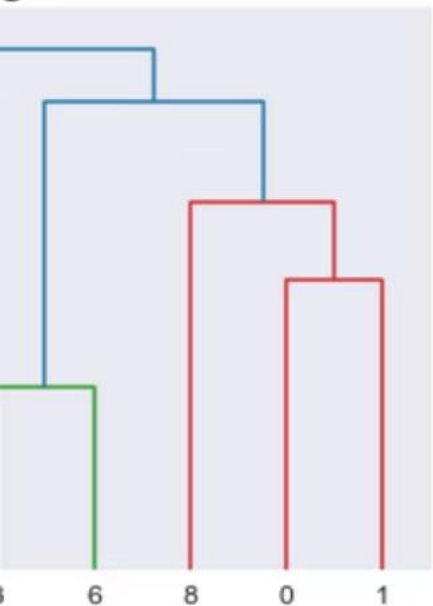
PCA, t-SNE para datos complejos



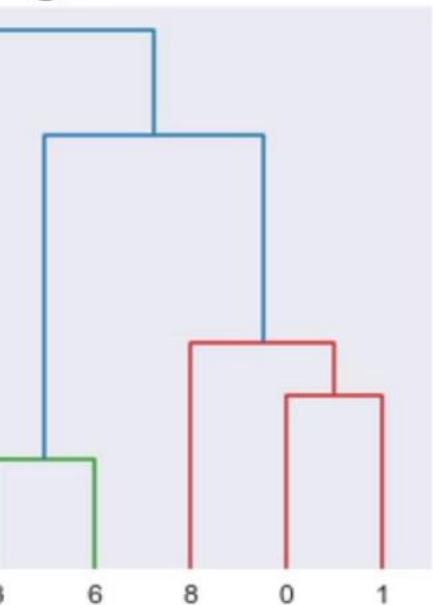
Coloración por cluster

Usar colores distintos para cada grupo identificado

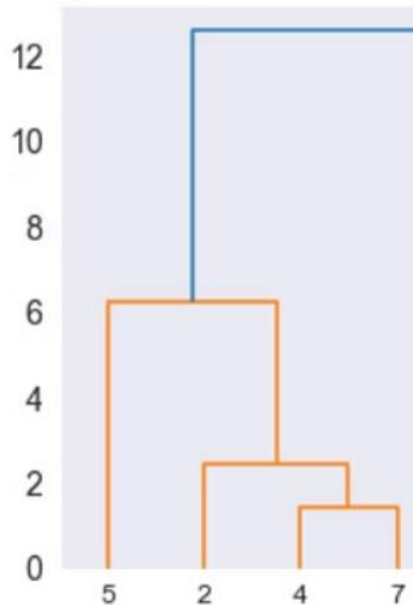
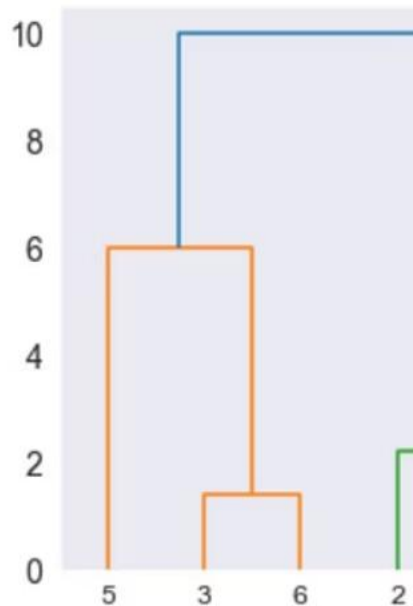
gle



age



C



Comparación de Métodos de Clustering

Método	Ventajas	Desventajas
K-Means	Rápido y eficiente	Requiere definir k previamente
Jerárquico	Muestra estructura de de datos	Computacionalmente pesado
DBSCAN	Detecta formas arbitrarias	Sensible a parámetros



K-MEANS

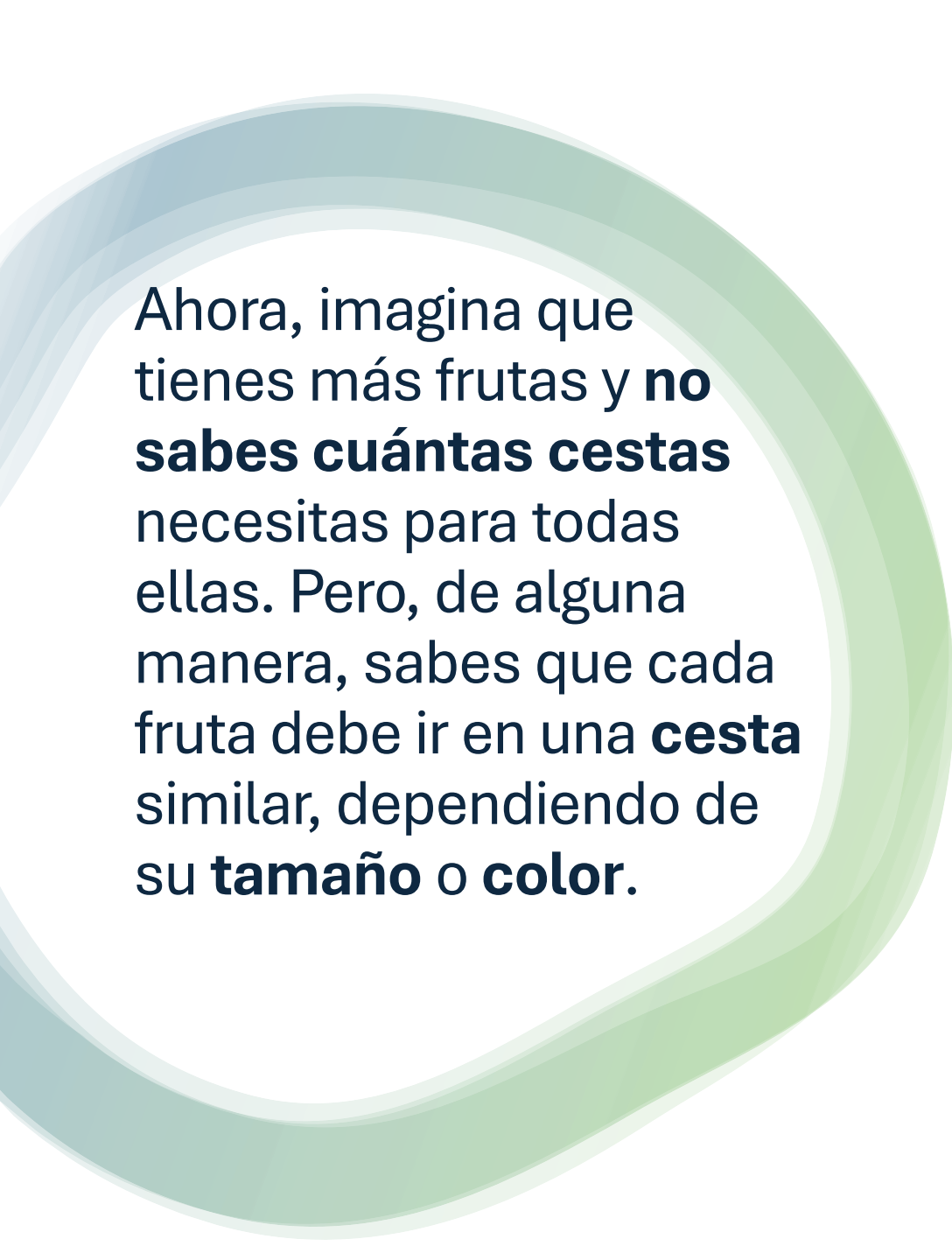


Imaginemos que estamos en un mercado, y tenemos un montón de **frutas** en una mesa. El objetivo es **agrupar** esas frutas en **cestas** basándonos en sus características (como su **tamaño** o **peso**). Pero aquí está la parte importante: **no sabemos cuántas cestas** necesitamos, y no tenemos etiquetas de qué fruta es cuál. Sin embargo, tenemos una idea de que las frutas similares deberían ir juntas.

Cómo lo haríamos probablemente.... :

1. **Mirar las frutas:** Ves una fruta, por ejemplo, una **manzana**. Te das cuenta de que es de **tamaño pequeño y roja**.
2. **Formar una cesta:** Pones la **manzana** en una **cesta**.
3. **Siguiente fruta:** Luego ves una **pera**. Dices: “Hmm, esta también es pequeña, y tiene una forma diferente, pero debería ir con la manzana”.
4. **Repetir el proceso:** Vas añadiendo más frutas a la **cesta de frutas rojas pequeñas**.





Ahora, imagina que tienes más frutas y **no sabes cuántas cestas** necesitas para todas ellas. Pero, de alguna manera, sabes que cada fruta debe ir en una **cesta** similar, dependiendo de su **tamaño o color**.

Lo que sucede en K-Means:

- **Las frutas:** Son tus **datos** (en este caso, las frutas tienen características como el precio en la granja y el precio en el mercado).
- **Las cestas:** Son los **clusters** que el algoritmo está creando. El algoritmo debe agrupar las frutas (datos) en **cestas** (clusters) similares.
- **¿Cuántas cestas?:** No sabes de antemano cuántas cestas (clusters) son necesarias, pero el algoritmo te ayudará a decidir. Es como si al principio no supieras cuántas cestas son necesarias, pero el algoritmo experimenta y encuentra el número ideal (por ejemplo, 3 cestas).
- **El centro de la cesta:** Cada **cesta** tiene un **centro** (el **centroide** en K-Means), que es como un **punto medio** de todas las frutas dentro de esa cesta. A medida que agregas más frutas, el **centro** de la cesta se mueve un poco, hasta que las frutas se agrupan correctamente

Pasos del algoritmo K- Means explicados de forma simple:

- 1. Decidir cuántas cestas necesitamos:** El algoritmo empieza decidiendo cuántas cestas quieres. Esto es algo que tú decides antes de comenzar, por ejemplo, si quieres 3 cestas, el algoritmo buscará **3 grupos**.
- 2. Inicializar las cestas:** El algoritmo comienza colocando **3 puntos aleatorios** en el espacio (esto sería como elegir 3 frutas al azar que van a ser el "centro" de cada cesta).
- 3. Asignar frutas a las cestas:** Ahora, el algoritmo ve todas las frutas (datos) y las coloca en la **cesta** más cercana al centro.
- 4. Mover los centros de las cestas:** Luego, el algoritmo **mueve** el centro de cada cesta al **promedio** de todas las frutas que hay dentro de esa cesta.
- 5. Repetir el proceso:** El algoritmo repite los pasos de asignar frutas a las cestas y mover los centros de las cestas, hasta que ya no se muevan más. Esto indica que el proceso ha **convergió**.

Analogía visual:



Imagina que tienes 3 **cestas vacías** y muchas **frutas en la mesa**.



Vas **llenando las cestas** con frutas, agrupándolas según **características similares** (tamaño, color, etc.).



Cada cesta se va llenando con frutas que son **similares entre sí**, y con el tiempo las cestas se ajustan para tener lo mejor de las frutas.



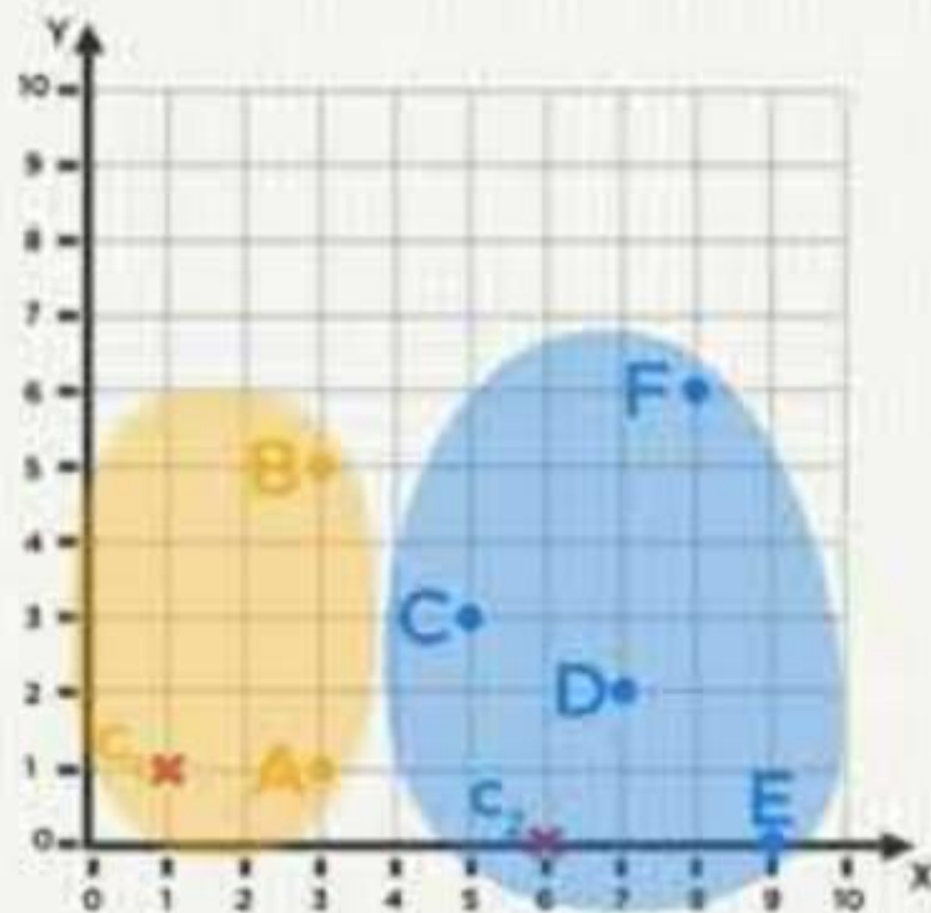
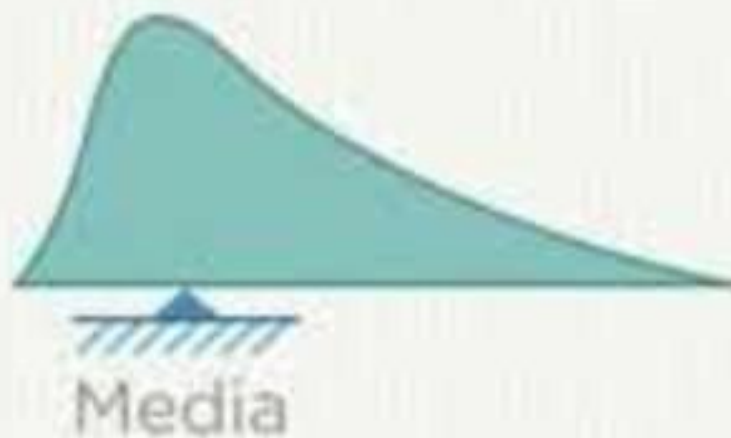
Después de algunas iteraciones, **todas las frutas estarán en las cestas correctas**, y tendrás **3 grupos** de frutas, según su **similitud**.

Algoritmo

C_1



C_2



Explicación del algoritmo:

1. **Tienes un montón de frutas (datos)** que necesitas agrupar. Algunas son rojas, otras son verdes, algunas son grandes y otras pequeñas. No sabes cuántos grupos de frutas necesitas.
2. **Le dices al algoritmo que haga el trabajo** y le pides que cree un número específico de **grupos** (por ejemplo, 3). Este número de grupos es lo que llamamos **clusters**.
3. El algoritmo comienza colocando **puntos aleatorios** en el espacio como el **centro** de cada grupo de frutas.
4. Luego, el algoritmo **mueve** esas frutas a las **cestas** más cercanas, basándose en la similitud.
5. El algoritmo ajusta las posiciones de las cestas y **repite** el proceso hasta que encuentra el **mejor agrupamiento posible**.



¿Por qué K-Means es útil?

- Es útil porque te permite descubrir **patrones** en los datos sin tener que etiquetar todo manualmente. El algoritmo encuentra la **estructura interna** de los datos, agrupando cosas que son **similares** entre sí.
- En el caso de tu dataset de frutas, K-Means te ayuda a **agrupar las frutas** basándote en sus **precios** sin tener que etiquetar manualmente qué tipo de fruta es cada una.



Resumen de K-Means:

- **K-Means es como agrupar frutas en cestas:** decides cuántas cestas quieres, y luego el algoritmo organiza las frutas según sus características.
- **No necesitas saber las etiquetas de los productos** (como si son frutas cítricas o no). El algoritmo solo agrupa las frutas según sus **similitudes**.
- **El número de cestas** (o clusters) debe ser decidido antes de empezar, pero el algoritmo ajustará las cestas (clusters) hasta que se agrupen correctamente.
- ¡Así que, K-Means es como un organizador de frutas, solo que en lugar de frutas, está organizando tus datos según sus características!



Diferencia entre K-Nearest Neighbors (KNN) y K-Means

K-Nearest Neighbors (KNN):

- **Objetivo:** KNN se utiliza para **clasificación** (aunque también puede usarse para regresión). El objetivo de KNN es **predecir la clase de un nuevo dato** basándose en las clases de los **K vecinos más cercanos**.
- **Requiere etiquetas:** Dado que es un algoritmo supervisado, necesitas **datos etiquetados**. Es decir, cada punto de tu dataset debe tener una etiqueta de clase (en tu caso, las frutas deberían estar categorizadas en tipos, como "cítricos", "hojas", etc.).
- **Aplicación en tu caso:** Para aplicar **KNN** en tu dataset de frutas, necesitas que los productos (frutas) estén etiquetados en categorías (por ejemplo, "cítricos", "vegetales", "raíces", etc.). El algoritmo usaría las características de los productos (como precios) para predecir su categoría.

- **Objetivo:** K-Means se utiliza para **clustering**, es decir, para **agrupar datos** en **clusters** o grupos.
- **Cómo funciona:**
 - K-Means asigna cada punto del dataset a un **centroide** de un cluster. Luego, ajusta los centroides iterativamente para minimizar la variabilidad dentro de cada cluster.
 - El número de clusters debe definirse de antemano, y los puntos se asignan al cluster más cercano en cada iteración.
- **No requiere etiquetas:** Dado que es un algoritmo no supervisado, no necesitas etiquetas previas. El algoritmo agrupa los puntos basándose solo en sus características.
- **Aplicación en tu caso:**

En este caso, al aplicar **K-Means** en el dataset de frutas, el algoritmo **no asigna etiquetas** sino que **agrupa las frutas** en diferentes clusters según la similitud de sus precios en los diferentes mercados.

K-Means:



Diferencias clave entre KNN y K-Means:

Aspecto	KNN (K-Nearest Neighbors)	K-Means
Tipo de algoritmo	Supervisado	No supervisado
Objetivo	Clasificación (o regresión)	Clustering (agrupamiento)
Requiere etiquetas	Sí, etiquetas de clase para cada punto	No, no requiere etiquetas
Salida	Asigna una etiqueta (clase) a cada punto nuevo	Asigna a cada punto un cluster (grupo)
Entrenamiento	No tiene fase de "entrenamiento" real, solo "aprende" de los vecinos	Realiza una fase de "entrenamiento" para encontrar los centroides de los clusters
Métrica utilizada	Distancia (normalmente Euclidiana)	Distancia a los centroides de los clusters
Ajuste de parámetros	El valor de K es el principal parámetro	El número de clusters (K) es el principal parámetro
Ejemplo de uso	Predecir el tipo de fruta basado en sus características (precios, tamaño, etc.)	Agrupar frutas según sus precios en diferentes mercados



Ejemplo: Aplicación de KNN vs. K-Means (mi dataset)

Con K-Nearest Neighbors (KNN):

- Imagina que en tu dataset de frutas, cada fruta tiene una categoría predefinida (por ejemplo, "cítricos", "verdes", "hojas", etc.).
 - Si una nueva fruta tiene precios similares a los de otras frutas de la categoría "cítricos", el algoritmo predice que esta fruta también es un **cítrico**.

Con K-Means:

- En K-Means, no tienes etiquetas, por lo que el algoritmo simplemente **agrupa las frutas en clusters** basados en la **similitud de precios**.
 - El resultado no será una categoría predicha, sino un grupo al que pertenece cada fruta.
 - **Cluster 1** podría incluir frutas con precios más bajos en la granja y en los mercados, mientras que **Cluster 2** podría incluir frutas con precios más altos en la granja pero precios más elevados en los mercados.



KNN se utiliza cuando tienes **etiquetas** predefinidas y quieres **predecir la categoría de nuevos datos**. Es un algoritmo de **clasificación supervisada**.

K-Means, por otro lado, es útil cuando **no tienes etiquetas** y deseas **agrupar los datos en clusters** basados en sus características. Es un algoritmo de **clustering no supervisado**.

```
[ ] import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Cargar el dataset
file_path = 'ProductPriceIndex.csv'
data = pd.read_csv(file_path)

# Limpiar los datos, eliminando el símbolo de dólar y convirtiendo a float
price_columns = ['farmprice', 'atlantaretail', 'chicagoretail', 'losangelesretail', 'newyorkretail']

# Limpiar las columnas de precios, eliminando caracteres no numéricos
for col in price_columns:
    data[col] = data[col].replace({'\$': '', ',': ''}, regex=True)

# Convertir a float y manejar errores si existen valores no convertibles
for col in price_columns:
    data[col] = pd.to_numeric(data[col], errors='coerce') # 'coerce' convierte a NaN los valores no válidos

# Manejar valores faltantes (NaN), si es necesario
data = data.dropna(subset=price_columns) # Eliminar filas con valores NaN en las columnas de precios

# Seleccionar solo las columnas de precios para el clustering
X = data[price_columns]

# Estandarizar los datos para que cada columna tenga media 0 y desviación estándar 1
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Aplicar el algoritmo K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
data['cluster'] = kmeans.fit_predict(X_scaled)

# Mostrar los resultados
plt.scatter(data['farmprice'], data['atlantaretail'], c=data['cluster'], cmap='viridis')
plt.xlabel('Farm Price')
plt.ylabel('Atlanta Retail Price')
plt.title('K-Means Clustering of Product Prices')
plt.colorbar(label='Cluster')
plt.show()

# Mostrar el dataframe con los clusters asignados
data[['productname', 'cluster']].head()

cluster_means = data.groupby('cluster')[price_columns].mean()
print(cluster_means)
```

```
# Aplicar el algoritmo K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
data['cluster'] = kmeans.fit_predict(X_scaled)
```

¿Qué hace KMeans?

El algoritmo **K-Means** es un algoritmo de **clustering no supervisado**. Su objetivo es agrupar los datos en un número predefinido de grupos o "clusters" basados en las similitudes entre los datos.

- **n_clusters=3**: Este parámetro indica que el algoritmo debe agrupar los datos en 3 clusters. Es un parámetro importante y depende de la cantidad de grupos que consideremos lógicos en el problema.
- **random_state=42**: Fija la semilla aleatoria para garantizar que los resultados sean reproducibles. Es útil si deseas obtener siempre el mismo resultado al ejecutar el código varias veces.

¿Qué hace fit_predict(X_scaled)?

- **fit_predict(X_scaled)**: Este método realiza dos acciones:

1.fit: El modelo encuentra los "centroides" (puntos centrales) de los clusters a partir de los datos estandarizados.

2.predict: Después de ajustar el modelo a los datos, asigna a cada punto (producto en este caso) un cluster (grupo) basado en la cercanía a los centroides calculados.

El resultado se almacena en `data['cluster']`, donde cada producto tiene asignado un número de cluster.

El algoritmo K-Means de sklearn tiene varios parámetros que puedes ajustar, dependiendo de tus necesidades.

Parámetro	Descripción	Valor predeter
n_clusters	Número de clusters (grupos) que el algoritmo debe generar.	8
init	Método para inicializar los centroides de los clusters. Puede ser 'k-means++' (más eficiente) o 'random' para elegir centros aleatorios.	'k-means++'
n_init	Número de veces que se ejecutará el algoritmo con diferentes centroides iniciales. El mejor resultado se usará. Esto ayuda a evitar que el algoritmo quede atrapado en un mínimo local.	10
max_iter	Número máximo de iteraciones para una única ejecución del algoritmo. Si no se alcanza la convergencia antes de esto, el algoritmo se detiene.	300
tol	Tolerancia para la convergencia. Si la diferencia en la posición de los centroides entre iteraciones consecutivas es menor que este valor, el algoritmo termina.	1e-4
precompute_distances	Indica si se debe precomputar las distancias. Puede ser útil para mejorar el rendimiento, pero no siempre es necesario.	'auto'
verbose	Si es mayor que 0, muestra información de progreso.	0
random_state	Semilla aleatoria para la inicialización de los centroides y la aleatoriedad en la ejecución. Esto asegura que los resultados sean reproducibles.	None
algorithm	Algoritmo utilizado para calcular los clusters: 'auto', 'full', 'elkan'. El algoritmo 'elkan' puede ser más rápido para ciertos tipos de datos.	'auto'
copy_x	Si se debe hacer una copia de los datos o trabajar directamente sobre ellos.	True
n_jobs	Número de trabajos paralelos que se usarán para el cálculo. -1 usa todos los procesadores disponibles.	1

Parámetros más importantes:

- 1.n_clusters:** Es el parámetro más importante, ya que define cuántos grupos queremos generar en el conjunto de datos.
- 2.init:** Define cómo se inicializan los centroides. Usar 'k-means++' generalmente produce mejores resultados en términos de tiempo de convergencia y calidad.
- 3.n_init:** Asegura que el algoritmo ejecute múltiples inicializaciones de los centroides para mejorar las probabilidades de encontrar una solución óptima.
- 4.max_iter:** Si los datos son grandes o difíciles de agrupar, puedes ajustar este parámetro para permitir que el algoritmo haga más iteraciones antes de detenerse.
- 5.tol:** Controla cuándo se detiene el algoritmo, basado en la convergencia. Si es un valor pequeño, el algoritmo se ejecutará más tiempo.

¿Cuándo ajustarlos?

- **n_clusters** se ajusta según cuántos grupos creas que deben existir en los datos. Puedes usar métodos como el **Codo** o **Silhouette Score** para decidir el número óptimo de clusters.
- **n_init** es útil para mejorar la precisión del algoritmo, pero no es crítico para la mayoría de los casos.
- **max_iter** y **tol** solo necesitan ser ajustados si estás tratando con datos complejos y no logras que el algoritmo converja rápidamente.

```
# Mostrar los resultados
plt.scatter(data['farmprice'], data['atlantaretail'], c=data['cluster'], cmap='viridis')
plt.xlabel('Farm Price')
plt.ylabel('Atlanta Retail Price')
plt.title('K-Means Clustering of Product Prices')
plt.colorbar(label='Cluster')
plt.show()

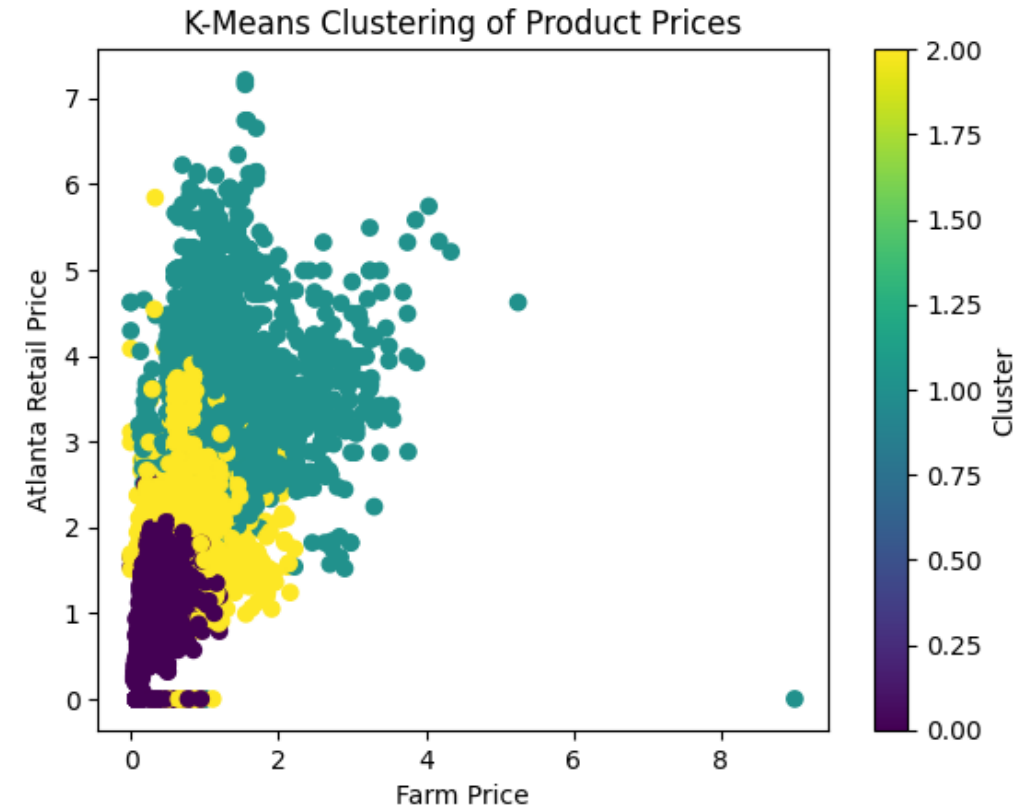
# Mostrar el dataframe con los clusters asignados
data[['productname', 'cluster']].head()
```

¿Qué hace este código?

- **plt.scatter**: Crea un gráfico de dispersión (scatter plot) con los datos proporcionados.
 - El eje X muestra los precios de **farmprice**.
 - El eje Y muestra los precios de **atlantaretail**.
- **c=data['cluster']**: Colorea los puntos según el número de cluster asignado (almacenado en data['cluster']). Esto permite visualizar cómo se agrupan los puntos. Los puntos que están en el mismo cluster tendrán el mismo color.
- **cmap='viridis'**: Define el esquema de color utilizado en el gráfico. viridis es una paleta de colores que va desde colores oscuros a colores más brillantes, lo cual facilita distinguir los grupos.

1. Colores (Clusters):

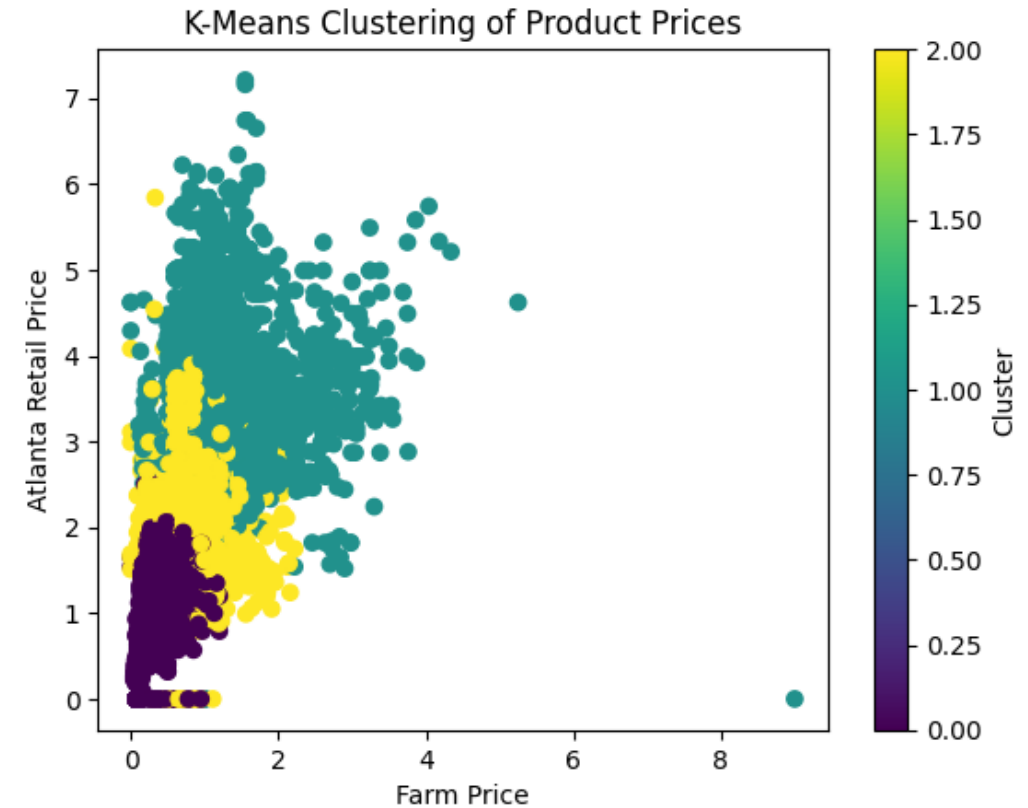
1. Los puntos están coloreados según los clusters asignados por el algoritmo K-Means. El **color** indica a qué cluster pertenece cada producto, según su comportamiento de precios en los mercados (precio en la granja y precio al por menor en Atlanta).
2. **Cluster 0:** Representado por el color **morado**. Estos productos parecen tener precios bajos tanto en la granja como en el mercado al por menor.
3. **Cluster 1:** Representado por el color **verde**. Estos productos probablemente tienen una variabilidad de precios mayor entre la granja y el mercado, y los precios al por menor en Atlanta son generalmente más altos.
4. **Cluster 2:** Representado por el color **amarillo**. Este cluster tiene productos con precios en la granja relativamente bajos, pero los precios al por menor en Atlanta parecen ser más altos, lo que sugiere que estos productos pueden tener un margen de ganancia significativo entre la granja y el mercado.



Explicación del gráfico:

2. Distribución de los puntos:

- Los productos de **Cluster 0 (morado)** parecen concentrarse en el extremo inferior izquierdo del gráfico, donde tanto el precio en la granja como el precio en Atlanta son bajos.
- Los productos de **Cluster 1 (verde)** están distribuidos a lo largo del gráfico, indicando una mayor variabilidad en los precios al por menor y en la granja.
- Los productos de **Cluster 2 (amarillo)** están principalmente en el centro del gráfico, con precios en la granja bajos y precios al por menor más altos, lo que sugiere que estos productos podrían tener márgenes de ganancia más grandes.



Explicación del gráfico:

Acciones sugeridas:



Análisis de Márgenes: Investigar más sobre las frutas o productos en cada cluster para entender mejor los márgenes de ganancia entre la granja y el mercado, especialmente para los productos del **Cluster 2 (amarillo)**.



Comparación con la demanda: Analizar cómo la demanda de estos productos varía en relación con sus precios y márgenes de ganancia. Por ejemplo, los productos con márgenes más altos (Cluster 2) podrían ser aquellos que tienen una demanda más estable o que son percibidos como productos premium.

```
cluster_means = data.groupby('cluster')[price_columns].mean()  
print(cluster_means)
```

data.groupby('cluster'):

- Esto agrupa el dataframe data en función de la columna cluster. Cada grupo estará compuesto por los datos correspondientes a un **cluster** distinto. Es decir, si tienes tres clusters (0, 1, 2), el dataframe se divide en tres grupos, uno para cada valor único en la columna cluster.

[price_columns]:

- Esto selecciona solo las columnas que están en la lista price_columns. En tu caso, price_columns es una lista de columnas como: price_columns = ['farmprice', 'atlantaretail', 'chicagoretail', 'losangelesretail', 'newyorkretail']

- **.mean():** El método .mean() calcula el promedio de cada columna de precios dentro de cada grupo (cluster). En otras palabras, para cada cluster, se calcula el valor promedio de los precios en las granjas y en los mercados de las frutas (según las columnas seleccionadas en price_columns).

productname	farmprice	atlantaretail	chicagoretail	losangelesretail	newyorkretail	cluster
Apple	0.50	1.00	1.10	1.05	1.20	0
Banana	0.60	1.20	1.15	1.25	1.30	0
Orange	1.00	2.00	2.10	2.05	2.20	1
Grapes	1.10	2.50	2.40	2.60	2.80	1
Pear	0.70	1.30	1.35	1.40	1.50	2

El resultado de cluster_means podría ser algo como esto:

cluster	farmprice	atlantaretail	chicagoretail	losangelesretail	newyorkretail
0	0.55	1.10	1.125	1.15	1.25
1	1.05	2.25	2.25	2.325	2.50
2	0.70	1.30	1.35	1.40	1.50

¿Por qué es útil este análisis?

- **Promedio por cluster:** Este análisis permite ver cómo se distribuyen los precios dentro de cada cluster. Por ejemplo, si un cluster tiene precios significativamente más altos que otro, podemos inferir que ese cluster puede estar compuesto por productos de **mayor calidad** o **productos premium**.
- **Toma de decisiones:** Con esta información, se pueden tomar decisiones estratégicas. Si un grupo de productos tiene precios muy similares en la granja y el mercado, se puede ajustar la estrategia de precios o marketing de manera más precisa.

```
cluster_means = data.groupby('cluster')[price_columns].mean()  
print(cluster_means)
```

Cluster	Farm Price	Atlanta Retail	Chicago Retail	Los Angeles Retail	New York Retail
0	0.310	1.087	1.021	1.041	1.251
1	1.341	3.507	3.404	3.432	3.685
2	0.605	1.779	1.786	1.768	2.061

Con mi escenario, este resultado muestra los **precios promedio** de cada cluster en los diferentes mercados (farmprice, atlanta retail, chicago retail, los angeles retail, y new york retail). Los valores son los promedios de los precios de los productos dentro de cada cluster, calculados después de haber aplicado el algoritmo K-Means.



Interpretación:

Cluster 0:

- **Farm Price (0.310):** Este cluster tiene precios muy bajos en la granja, lo que podría indicar productos básicos o de bajo costo.
- **Retail Prices:** Los precios en los mercados (Atlanta, Chicago, Los Ángeles, Nueva York) son también relativamente bajos en comparación con los otros clusters. Esto sugiere que estos productos no tienen un gran margen de ganancia o que son productos de consumo masivo que no tienen una gran diferencia de precio entre la granja y el mercado.

Cluster 1:

- **Farm Price (1.341):** Los productos en este cluster tienen un precio significativamente más alto en la granja, lo que puede indicar que son productos más caros o de mayor calidad.
- **Retail Prices:** Los precios al por menor en todos los mercados (Atlanta, Chicago, Los Ángeles, Nueva York) también son mucho más altos. Esto podría implicar que estos productos tienen un mayor margen de ganancia o que son productos premium o de temporada.

Cluster 2:

- **Farm Price (0.605):** El precio en la granja es algo mayor que en el Cluster 0, pero aún bajo comparado con el Cluster 1.
- **Retail Prices:** Los precios al por menor están en un rango intermedio, ligeramente más altos que en el Cluster 0, pero no tan elevados como en el Cluster 1. Este grupo podría representar productos con un margen de ganancia moderado.

Conclusiones:



Cluster 0: Los productos de este cluster son de bajo precio, tanto en la granja como en los mercados. Son probablemente productos básicos con una demanda estable, pero con márgenes de ganancia bajos.



Cluster 1: Los productos de este cluster tienen precios más altos en la granja y en los mercados, lo que indica un mayor margen de ganancia. Podrían ser productos premium, de temporada o con características especiales que justifican su precio elevado.



Cluster 2: Los productos de este cluster tienen precios intermedios, lo que podría sugerir que estos productos tienen una mayor variabilidad en la oferta y demanda o que son productos que están en el medio de la cadena de valor, con márgenes de ganancia moderados.

Cluster 0 (Precios bajos):

Farm Price (0.310) y precios de **retail** relativamente bajos.

Esto sugiere que los productos en este cluster tienen precios similares tanto en la granja como en el mercado. Son probablemente **productos básicos** o de **consumo masivo**. Este tipo de productos a menudo tiene una **alta demanda**, pero los márgenes de ganancia son más bajos debido a su bajo precio.

Ejemplo: Productos como frutas y verduras comunes que se venden a precios competitivos en diferentes mercados.

- **Cluster 1 (Precios altos, margen alto):**

- **Farm Price (1.341)** y precios de **retail** significativamente más altos.
- Este cluster agrupa productos con precios elevados tanto en la granja como en los mercados. Los productos en este grupo tienen **márgenes de ganancia más altos**, ya que los precios de venta al por menor son considerablemente más altos que los de la granja.
- Estos productos pueden ser **premium**, de **alta calidad** o de **temporada**.
- **Ejemplo:** Productos especiales como frutas exóticas o productos orgánicos que se venden a precios más altos.

•Cluster 2 (Precios intermedios):

- Farm Price (0.605)** y precios de **retail** en un rango intermedio.
- Los productos de este cluster tienen precios más altos que los del Cluster 0, pero no tan altos como los del Cluster 1. Esto puede indicar que estos productos tienen un **margen de ganancia moderado** y podrían representar un **segmento de mercado intermedio**.
- Ejemplo:** Productos que se venden bien en mercados donde los consumidores están dispuestos a pagar un poco más por productos de calidad moderada, pero no están dispuestos a pagar precios premium.

Acciones sugeridas:



Análisis de Márgenes: Podrías analizar la **diferencia de precios** entre la granja y los mercados para cada cluster. Los productos del **Cluster 1** probablemente tengan los mayores márgenes de ganancia.



Segmentación de Productos: Los productos en el **Cluster 1** podrían ser segmentados como productos premium, mientras que los de **Cluster 0** podrían ser más adecuados para estrategias de precios bajos o mercados masivos.



Ajuste de Estrategias de Precios: Analizar estos clusters te ayudará a ajustar las estrategias de precios, especialmente si eres un productor o distribuidor de estos productos. Los productos del **Cluster 1** pueden justificar precios más altos, mientras que los del **Cluster 0** pueden necesitar una estrategia basada en precios competitivos.

1. Análisis de márgenes de ganancia:

- **¿Qué es esto?**

Los márgenes de ganancia se refieren a la diferencia entre el **precio de venta** (al por menor) y el **precio de producción** (en la granja). Al analizar estos márgenes, puedes identificar qué productos generan **mayores ganancias** en cada etapa de la cadena de valor.

- **Impacto de aplicar K-Means:**

Al aplicar K-Means, identificas **productos con altos márgenes** (Cluster 1) y **productos con márgenes bajos** (Cluster 0). Esto te permite enfocar tus esfuerzos en productos de mayor rentabilidad, mientras que para los de bajo margen, podrías explorar maneras de **optimizar costos** o mejorar la eficiencia de producción.

- **Acción recomendada:**

Analiza los productos en el **Cluster 1**, ya que tienen los **márgenes de ganancia más altos**. Si eres productor o distribuidor, puedes **optimizar tu estrategia de precios** para estos productos. Además, los productos en **Cluster 0** pueden beneficiarse de **estrategias de precios bajos** o de **optimización de procesos** para mejorar la rentabilidad.





2. Segmentación de productos para diferentes mercados:

- **¿Qué es esto?**
Al agrupar productos que tienen comportamientos de precios similares, puedes adaptar tu **estrategia de marketing y ventas** de manera más efectiva a cada grupo de productos.
- **Impacto de aplicar K-Means:**
El **Cluster 1** (productos de alto precio) puede ser dirigido a **mercados premium** o **nichos de alto poder adquisitivo**. Mientras tanto, el **Cluster 0** puede estar dirigido a **mercados masivos** o **precios competitivos**. Esto te permitirá personalizar las estrategias de precios, marketing y distribución según las características de cada grupo de productos.
- **Acción recomendada:**
Para los productos del **Cluster 1**, considera estrategias de **marca premium**, **publicidad dirigida** y enfoques en **segmentos de alto poder adquisitivo**. Para los productos del **Cluster 0**, las estrategias pueden centrarse en **precios bajos** y **alta disponibilidad** en grandes mercados.

3. Ajuste de estrategias de precios:

- **¿Qué es esto?**

Una vez que entiendes cómo se distribuyen los precios en los diferentes clusters, puedes **ajustar las estrategias de precios** para maximizar las ganancias y asegurar que cada grupo de productos esté bien posicionado en el mercado.

- **Impacto de aplicar K-Means:**

K-Means ayuda a identificar productos con **precios elevados** o **márgenes más grandes**, lo que facilita la **optimización de precios**. Por ejemplo, para los productos del **Cluster 1**, puedes aplicar precios más altos, mientras que para los productos del **Cluster 0**, puedes buscar estrategias para reducir costos y mantener precios bajos.

- **Acción recomendada:**

Puedes usar los resultados de K-Means para ajustar tus **estrategias de fijación de precios** para los productos de cada cluster. Asegúrate de que los productos del **Cluster 0** no estén sobrepreciados en mercados masivos y que los del **Cluster 1** estén correctamente posicionados como productos premium.



4. Análisis de oferta y demanda:

- **¿Qué es esto?**

Comprender la relación entre los **precios** y la **demanda** de productos puede ayudarte a optimizar la **gestión del inventario** y las **estrategias de venta**.

- **Impacto de aplicar K-Means:**

El **Cluster 0** podría tener una **demanda más estable**, ya que los productos con precios bajos suelen ser más consumidos. Por el contrario, los productos del **Cluster 1** podrían tener una **demanda más variable** basada en tendencias de mercado o estacionalidad.

- **Acción recomendada:**

Para los productos del **Cluster 1**, asegúrate de contar con una **oferta constante** de productos de alta calidad y **estrategias de marketing estacionales**. Para los productos del **Cluster 0**, es importante mantener un **flujo constante de suministro** a precios competitivos.

Resumen de lo que K-Means ayuda a lograr:

- **Agrupación de productos:** K-Means agrupa productos con características similares (precios), lo que permite segmentar mejor el mercado.
- **Optimización de precios y márgenes:** Analizando los clusters, puedes **ajustar estrategias de precios** para cada tipo de producto (premium vs. masivo).
- **Segmentación estratégica:** Identificar qué productos tienen **márgenes altos** y cuáles tienen **bajos** permite tomar decisiones más informadas sobre dónde concentrar esfuerzos y recursos.
- **Mejor comprensión de la oferta y demanda:** Al comprender cómo los precios afectan a la demanda, puedes **optimizar la distribución y la gestión del inventario**.
- El uso de K-Means te permite tomar decisiones más **informadas y precisas** sin necesidad de tener etiquetas predefinidas o información extra sobre los productos, lo que es una gran ventaja en el análisis **no supervisado**.

**¿Cómo saber qué
características
usar para el
clustering?**

1. Características relevantes para el problema

2. Tipos de características que puedes usar

3. Escalabilidad de los datos

4. Correlación entre características

1. Características relevantes para el problema

- **Relación con el objetivo:** Antes de aplicar K-Means o cualquier algoritmo de clustering, debes pensar en el objetivo de tu análisis. ¿Qué características o variables son relevantes para el problema que estás tratando de resolver?
 - **Ejemplo en mi caso:** Como estoy trabajando con precios, las características más relevantes son los **precios en diferentes mercados**. Pero si mi objetivo fuera clasificar las frutas en grupos según su **tamaño**, también podría incluir el **tamaño** o **peso** de las frutas como características para el clustering.
- **Características que pueden representar diferencias entre grupos:** Las características que elijas deben ser capaces de **diferenciar** los grupos que buscas identificar. Si todas las frutas tienen precios similares en todos los mercados, el clustering no va a ser muy efectivo. Si tienes **diferencias claras** entre las frutas en términos de **precio, tamaño o calidad**, esas son buenas características para el clustering.

2. Tipos de características que puedes usar:

- **Características numéricas:** Son las más fáciles de usar porque pueden ser directamente alimentadas al modelo. Algunas características numéricas comunes son:
 - **Precios:** Como en mi caso.
 - **Peso o tamaño** de los productos.
 - **Cantidad de ventas.**
 - **Valor de propiedades** (por ejemplo, en un dataset inmobiliario, el valor de la propiedad).

2. Tipos de características que puedes usar:

- **Características categóricas:** Si tienes datos categóricos (como "color", "tipo de producto", "región"), debes **convertirlos en variables numéricas** antes de utilizarlos en el clustering. Esto se puede hacer usando técnicas como:
 - **One-hot encoding:** Convierte una columna categórica en varias columnas binarias (0 o 1). Ejemplo: "Fruta" podría convertirse en "Manzana", "Pera", "Plátano", etc.
 - **Label encoding:** Asigna un número entero a cada categoría. Ejemplo: "Rojo" = 1, "Verde" = 2.
 - **Combinación de características:** A veces, puedes crear nuevas características combinando otras. Por ejemplo:
 - **Precio relativo:** Puedes dividir el precio de un producto en diferentes mercados por el precio en la granja para obtener una medida del **margen de ganancia relativo**.
 - **Promedio de precios:** Crear una nueva característica que sea el **promedio de precios** en todos los mercados.

3. Escalabilidad de los datos:

Recordar que K-Means es **sensible a la escala** de los datos. Las características con valores muy grandes (como precios en grandes cantidades) pueden dominar la asignación de clusters. Por eso es importante **escalar los datos** antes de aplicar K-Means (como se ha hecho con StandardScaler).

4. Correlación entre características

- Si las características están **fuertemente correlacionadas**, podrías considerar **eliminar algunas de ellas** para evitar redundancias. Si tienes varias columnas que básicamente están midiendo lo mismo (por ejemplo, precios en diferentes mercados), podrías intentar usar **Análisis de Componentes Principales (PCA)** para reducir la dimensionalidad y mantener solo las características más relevantes.

Ejemplo práctico: ¿Qué características escoger?

Supongamos que tienes un dataset con frutas y quieres agruparlas según **sus precios y tamaños**. Podrías tener las siguientes columnas:

- **Precio en la granja**
- **Precio en diferentes mercados**
- **Tamaño de la fruta**
- **Peso de la fruta**
- **Tipo de fruta (Cítrico, Fruta de hueso, etc.)**



Pasos para elegir qué usar

:

1.Características numéricas: Primero selecciona las columnas que son **numéricas**, como los precios de la fruta y el tamaño o peso. Estas son fáciles de usar en el clustering.

- Por ejemplo: farmprice, atlantaretail, chicagoretail, weight, size.

2.Transformar características categóricas: Si tienes alguna columna **categórica** como type_of_fruit (por ejemplo, "Cítrico", "Baya", "Manzana"), primero debes transformarla a **números** usando **One-hot encoding** o **Label encoding**.

3.Combinación de características: Si crees que ciertas características podrían ser más útiles combinadas (por ejemplo, precio y tamaño), puedes crear una nueva característica que combine estos dos factores.

4.Evaluación: Si no estás seguro de qué características usar, puedes probar diferentes combinaciones y ver cuál produce el mejor clustering. Por ejemplo, podrías hacer pruebas de agrupamiento solo con precios (farmprice, atlantaretail, etc.), o también incluir el **tamaño** y **peso** de las frutas.

En resumen:

Selecciona las características que sean relevantes para el problema que estás resolviendo.

Usa características numéricas directamente, y convierte las categóricas en números si las vas a usar.

Asegúrate de escalar los datos si es necesario, para que las características no dominen el clustering debido a diferencias de escala.

Si no estás seguro de qué características usar, prueba diferentes combinaciones y evalúa qué resultado te da el mejor agrupamiento.



¿Cómo saber si el clustering tiene sentido?
existen técnicas o algo para esto?

¿Cómo saber si el clustering tiene sentido?

Una vez que has hecho el clustering, es importante que los resultados tengan **sentido lógico**. Por ejemplo:

- Si agrupas frutas por precios, ¿tienen sentido los grupos generados? ¿El grupo de frutas con precios bajos tiene un perfil coherente? ¿Y el grupo con precios altos?
- Si agrupas frutas por **peso y tamaño**, ¿las frutas con características similares se agrupan bien juntas?
- Si los resultados no tienen sentido, puedes intentar ajustar las características seleccionadas o probar con un número diferente de clusters.

Determinar si el clustering tiene sentido o si los clusters generados por el algoritmo son **útiles** es un paso crítico en cualquier análisis de clustering. Aunque el **clustering** es un proceso **no supervisado** (es decir, no tenemos etiquetas de clase para evaluar directamente los resultados), existen varias **técnicas** y **métricas** que nos permiten **evaluar la calidad** y la **coherencia** de los clusters obtenidos.



1. Análisis Visual

Una de las formas más simples y efectivas de evaluar los resultados del clustering es **visualizarlos**. Si los datos son de alta dimensión (como en tu caso, con múltiples mercados), puedes utilizar **técnicas de reducción de dimensionalidad** para proyectar los datos en 2 o 3 dimensiones y analizar la distribución de los clusters.

Técnicas de reducción de dimensionalidad:

- **PCA (Análisis de Componentes Principales):** Esta técnica reduce las dimensiones de los datos, encontrando las combinaciones lineales de características que capturan la mayor parte de la varianza en los datos. Puedes visualizar los clusters en 2D o 3D después de aplicar PCA.
- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** Esta técnica también reduce la dimensionalidad, pero lo hace preservando la **proximidad local** entre los puntos. Es muy útil cuando quieres mantener la estructura de los grupos a pequeña escala.

¿Qué es PCA?

PCA (Principal Component Analysis) es una técnica de reducción de dimensionalidad que:

- Transforma tus variables originales en un nuevo conjunto de variables llamadas **componentes principales**.
- Estos componentes son **combinaciones lineales** de las variables originales.
- Se ordenan según cuánta **varianza (información)** conservan del conjunto original:
 - **Primer componente:** contiene la mayor parte de la varianza.
 - **Segundo componente:** la siguiente mayor, y así sucesivamente.

Objetivo: proyectar los datos en 2D o 3D para visualizar estructuras como clústers, sin perder demasiada información.

- Cada punto representa un producto, proyectado en el plano de los dos primeros componentes principales.
- Los colores representan a qué clúster pertenece cada punto según K-Means.
- **Si ves grupos compactos y bien separados**, el valor de k elegido tiene sentido.
- **Si hay solapamiento visual**, puede que k no sea el ideal o que los datos no sean naturalmente separables.

```
# 4. Reducir a 2D con PCA
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
# 5. Visualizar los clústers
```

```
plt.figure(figsize=(8,5))
```

```
plt.scatter(X_pca[:, 0], X_pca[:, 1])
```

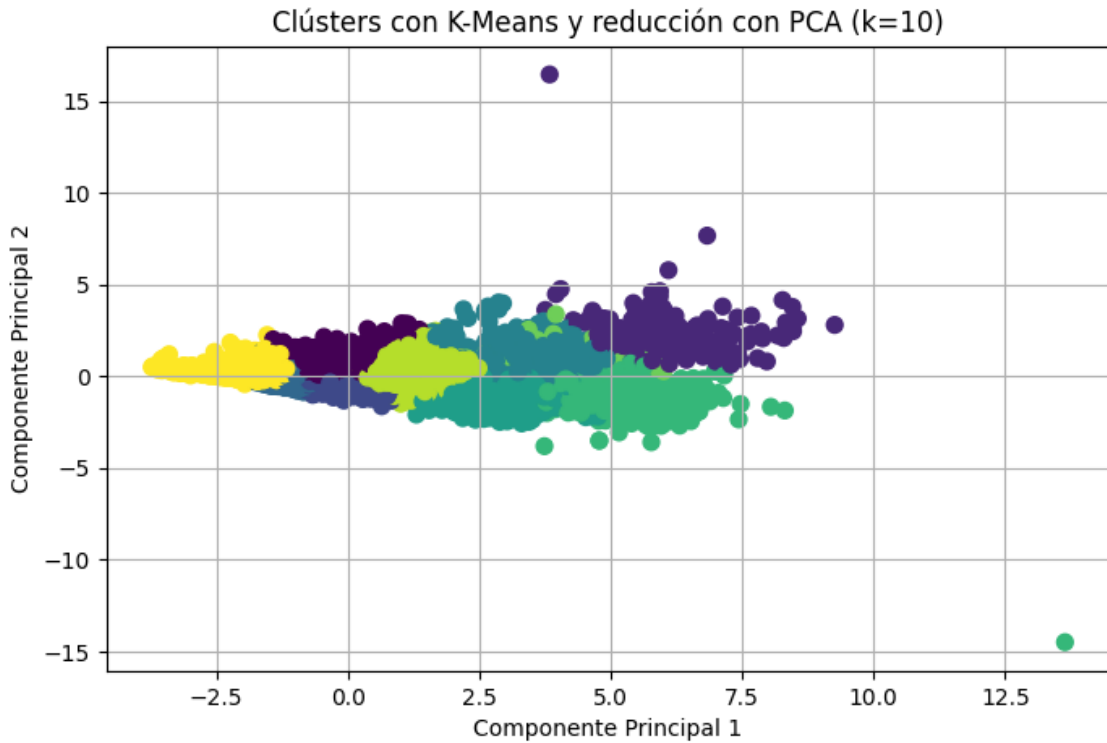
```
plt.title(f"Clústers con K-Means y PCA")
```

```
plt.xlabel("Componente Principal 1")
```

```
plt.ylabel("Componente Principal 2")
```

```
plt.grid(True)
```

```
plt.show()
```



¿Qué representa esta gráfica?

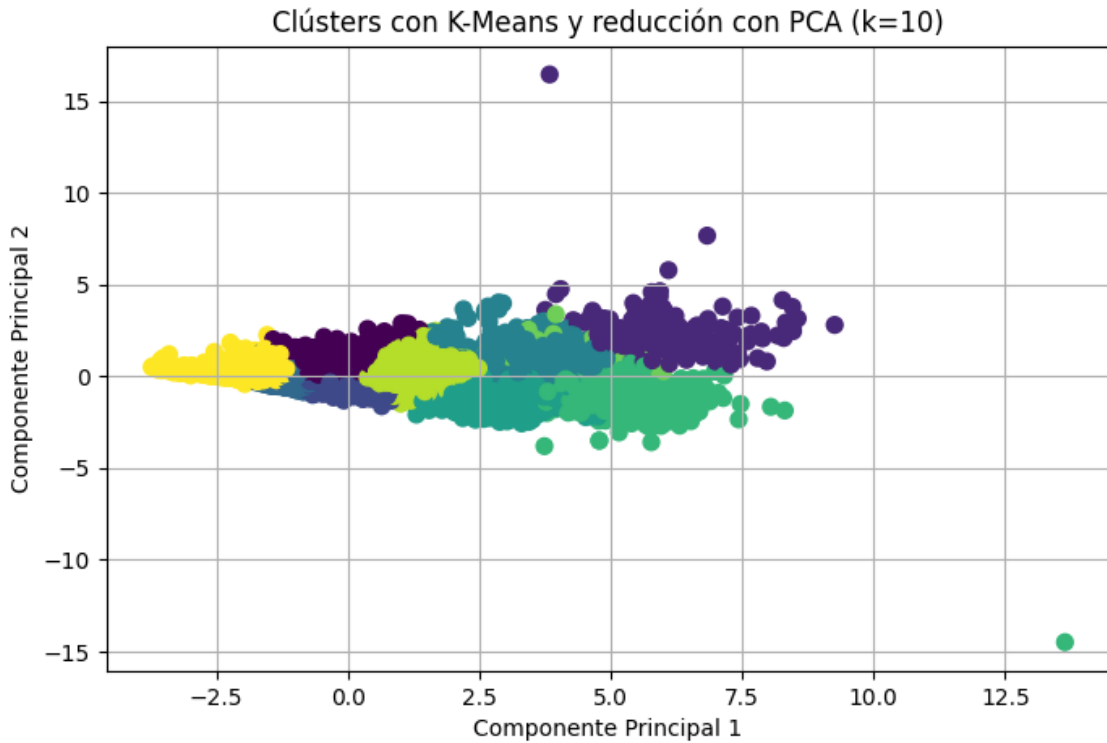
- **Ejes:**

- *Componente Principal 1 y Componente Principal 2* son combinaciones de tus variables originales (price_usd, discount, shipping_cost).

- Estos dos ejes explican la mayor parte de la **varianza** de los datos originales.

- **Puntos:** cada uno representa un producto.

- **Colores:** indican a qué clúster fue asignado cada producto por **K-Means con $k = 10$** .



¿Qué observamos?

1. Distribución alargada en el eje horizontal (PC1):

Esto sugiere que la **mayor parte de la varianza está capturada por el primer componente** (e.g., probablemente una combinación del precio total).

2. Mucho solapamiento visual entre clústers:

No hay una separación clara entre muchos de los colores. Algunos clústers se mezclan bastante → **K = 10 parece ser demasiado alto**.

3. Clústers poco compactos y dispersos:

Algunos puntos están muy alejados del grupo general, indicando **posibles outliers o mala asignación de clústers**.

¿Qué significa esto?

El resultado **muestra**: usar **demasiados clústers no tiene sentido** para estos datos.

Aunque K-Means fuerza a crear 10 clústers, los datos **no tienen 10 grupos naturales bien definidos**.

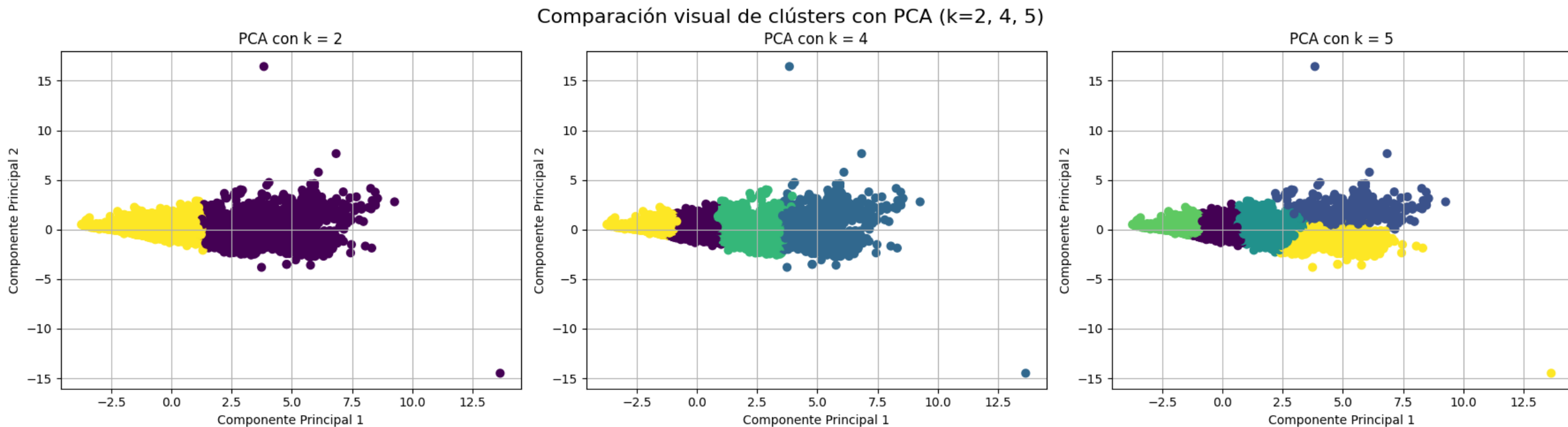
Es probable que **2 o 4 clústers** sean más apropiados.

```
# 4. Graficar con diferentes k en subplots
k_values = [2, 4, 5]
fig, axes = plt.subplots(1, len(k_values), figsize=(18, 5))

for i, k in enumerate(k_values):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
    labels = kmeans.fit_predict(X_scaled)

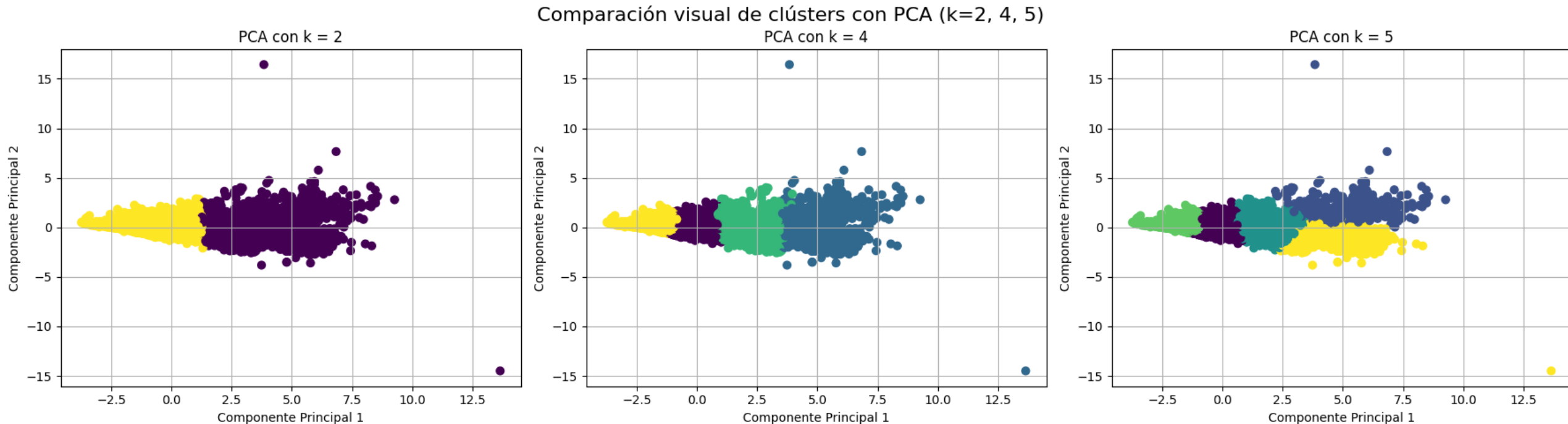
    axes[i].scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', s=40)
    axes[i].set_title(f'PCA con k = {k}')
    axes[i].set_xlabel("Componente Principal 1")
    axes[i].set_ylabel("Componente Principal 2")
    axes[i].grid(True)

plt.suptitle("Comparación visual de clústers con PCA (k=2, 4, 5)", fontsize=16)
plt.tight_layout()
plt.subplots_adjust(top=0.88)
plt.show()
```



¿Qué representa esta visualización?

- Cada gráfico muestra los datos reducidos a **2 componentes principales (PCA)**.
- Los puntos están coloreados según la asignación de **clústers por K-Means**.
- Esta proyección permite observar cómo se distribuyen y solapan los clústers en un espacio que conserva la mayor varianza posible de los datos originales.



Análisis por cada valor de k

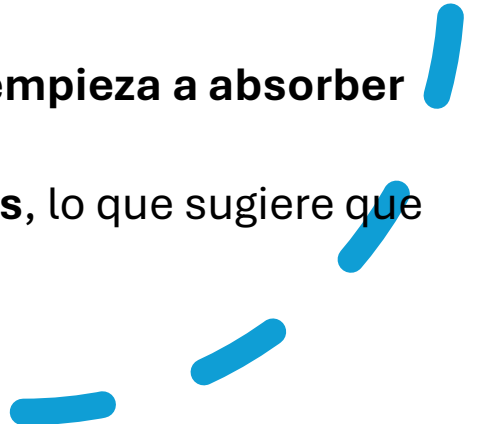
♦ $k = 2$

- Hay una **división clara** entre dos regiones a lo largo del **Componente Principal 1**.
- **Buen contraste** entre grupos, especialmente en la zona izquierda.
- Este resultado **refuerza lo visto en Silhouette y Dunn Index**, donde $k = 2$ fue fuerte.

♦ $k = 4$

- Se observa una **mayor granularidad**, con algunos subgrupos apareciendo claramente.
- Aunque hay cierto solapamiento en la zona media, se distinguen **nuevas estructuras** en la parte derecha.
- Este valor de k ofrece un **buen balance entre claridad visual y segmentación útil**.

♦ $k = 5$

- Aparecen más colores, pero **el clúster central empieza a absorber muchos puntos**, perdiendo claridad.
 - Algunos grupos están **mal definidos o dispersos**, lo que sugiere que K-Means está forzando una división artificial.
 - **Poco valor agregado respecto a $k = 4$.**
- 

Conclusión

Con **k = 4** vuelve a mostrarse como un **punto ideal** de equilibrio entre separación, detalle y estabilidad visual.

k	¿Separación clara?	¿Estructura útil?	¿Sobrecarga visual?	Recomendación
2	✓ Alta	● Muy general	✗ Ninguna	✓ Para segmentación binaria
4	✓ Buena	✓ Detallada	● Manejable	★ Mejor balance
5	● Regular	● Confusa	✓ Alta	✗ Evitar si no es necesario

¿Qué es t-SNE?

t-SNE (t-distributed Stochastic Neighbor Embedding) es un algoritmo de reducción de dimensionalidad **no lineal** que:

- Preserva **las relaciones locales** entre los puntos.
- Es ideal para visualizar datos de alta dimensión en **2D o 3D**.
- A diferencia de PCA (lineal), t-SNE puede descubrir **estructuras complejas y curvas** en los datos.

¿Cuándo usar t-SNE?

- Cuando los clústers en PCA **se solapan mucho**.
- Cuando tus datos tienen relaciones **no lineales**.
- Para **visualización** (no se recomienda para entrenamiento de modelos).

Consideraciones antes de usar t-SNE

- Es **computacionalmente costoso** (más lento que PCA).
- Los resultados pueden **variar entre ejecuciones**, porque es estocástico.
- **No se puede interpretar directamente como PCA** (las distancias no son absolutas, solo relativas).

Parámetros clave de t-SNE

```
# 4. Reducir a 2D con t-SNE
tsne = TSNE(n_components=2, perplexity=30, learning_rate=200, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# 5. Visualización
plt.figure(figsize=(8, 5))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=labels, cmap='viridis', s=50)
plt.title(f'Clústers visualizados con t-SNE (k={k})')
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2')
plt.grid(True)
plt.show()
```

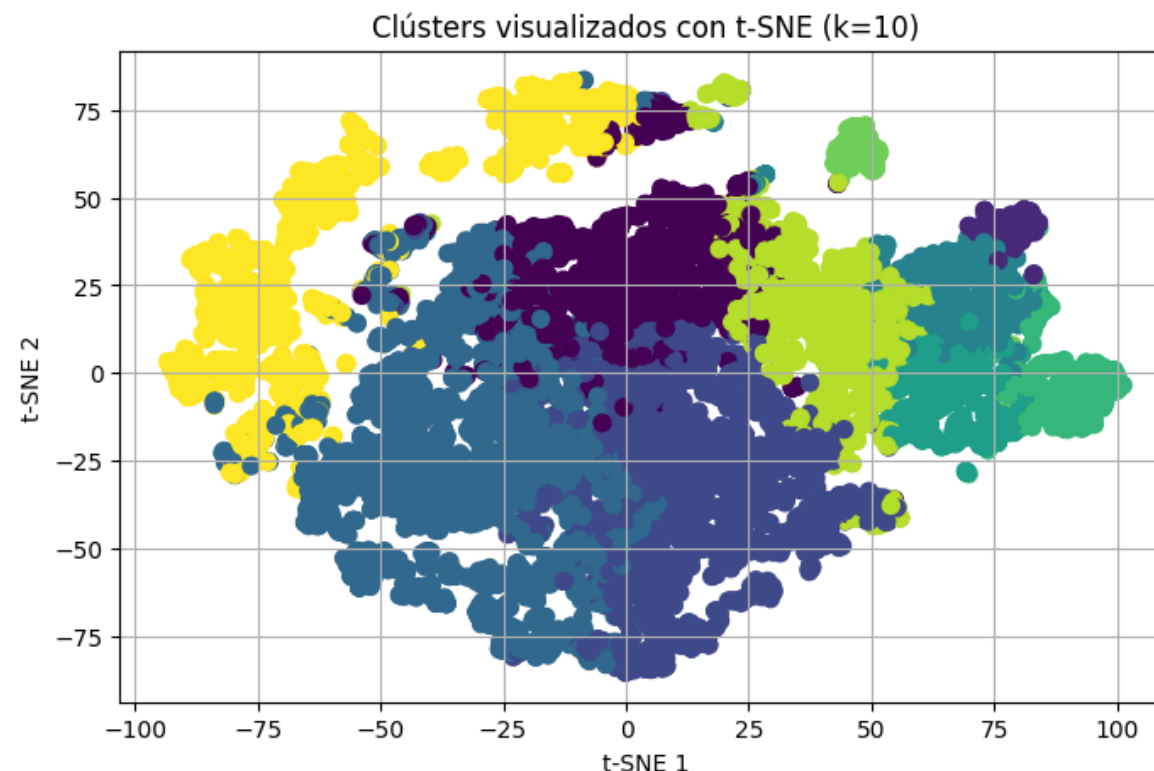
Parámetro	Significado	Recomendación
perplexity	Equilibrio entre enfoque local y global	Prueba entre 5 y 50
learning_rate	Tamaño del paso de aprendizaje	100–1000, comúnmente 200
random_state	Para resultados reproducibles	Usa uno fijo (ej. 42)

¿Qué observar?

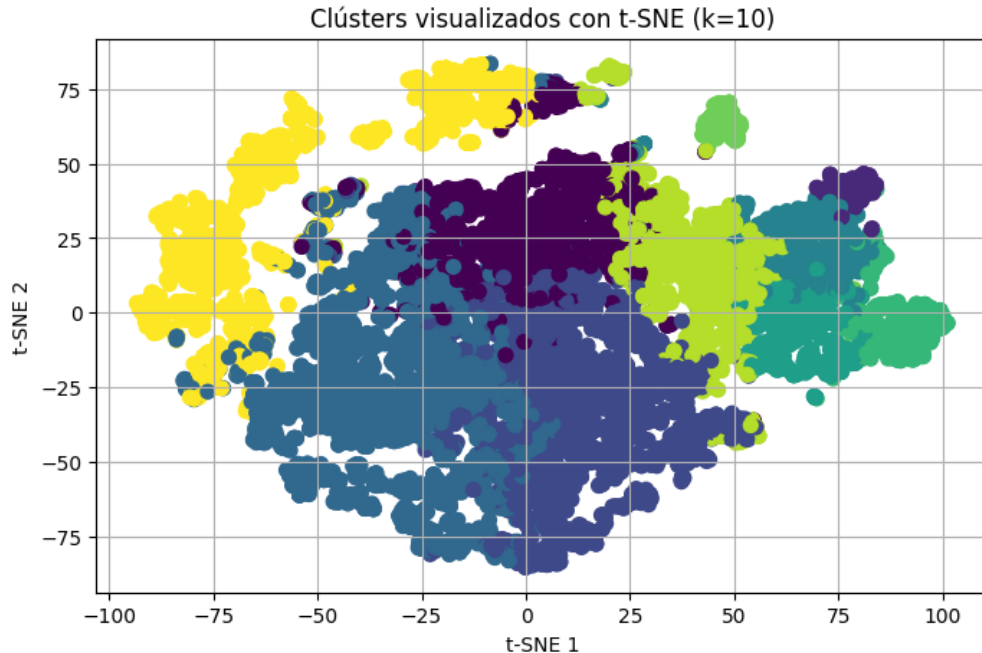
- **Grupos bien separados** → clústers bien definidos.
- **Grupos enredados o muy juntos** → quizás el valor de *k* es demasiado alto.
- A diferencia de PCA, t-SNE puede mostrar **formas curvas o ramificadas**, lo cual es normal.

¿Qué muestra esta gráfica?

- Cada punto representa un producto, reducido a 2D con **t-SNE**.
- Los colores indican a qué clúster lo asignó **K-Means con k=10**.
- Los ejes t-SNE 1 y t-SNE 2 **no tienen significado absoluto**: solo indican cómo se relacionan los puntos **entre sí**.



Interpretación visual



1. Separación parcial de clústers:

- Algunos colores se agrupan bien (como el amarillo, verde claro y verde esmeralda).
- Otros clústers (colores oscuros) **se mezclan fuertemente** en el centro → **solapamiento alto**.

2. Concentración central:

- Hay una **zona central muy densa** con muchos colores mezclados, lo que indica que K-Means está **forzando clústers** donde no hay separación clara.

3. Clústers mal definidos:

- El resultado refuerza lo que ya viste en **Silhouette Score** y **Dunn Index**:
→ **k=10 no tiene sentido**, hay **poco valor en tener tantos clústers**.

Métrica	Resultado con k=10	Conclusión
PCA	Clústers solapados	×
t-SNE	Grupos mezclados y centrales	×
Silhouette	Muy bajo para k=10	×
Dunn Index	Muy bajo para k=10	×

```
# 3. Reducir una sola vez con t-SNE (para comparar en el mismo espacio)
tsne = TSNE(n_components=2, perplexity=30, learning_rate=200, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

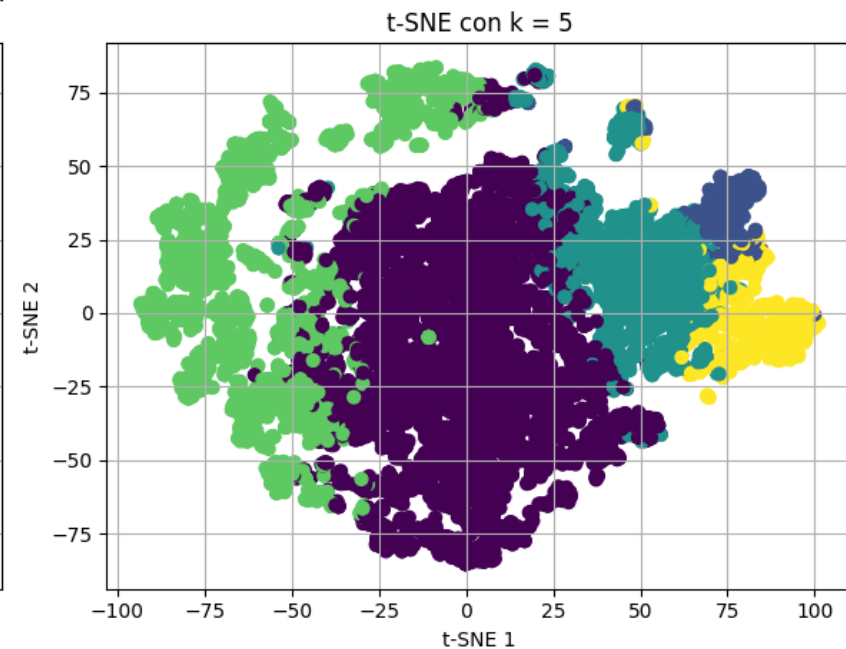
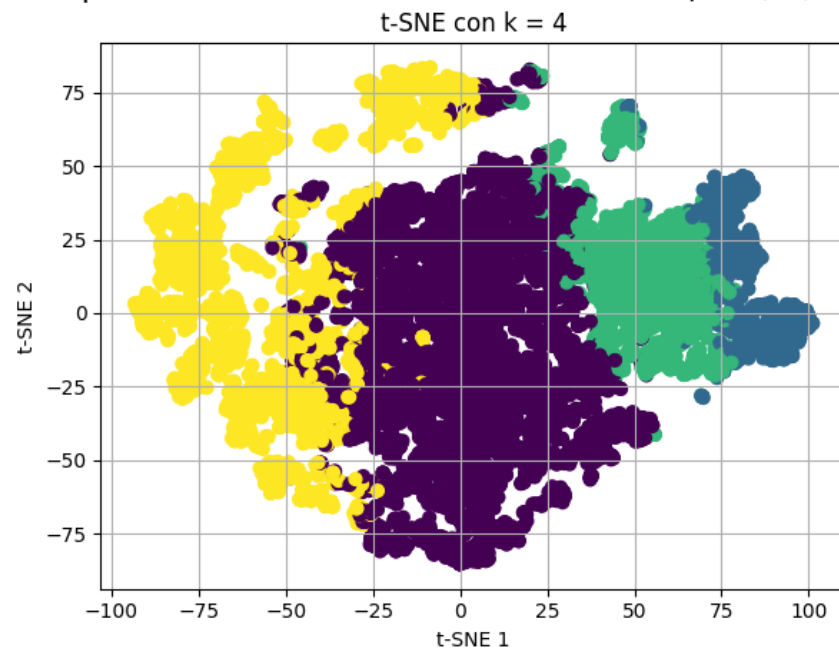
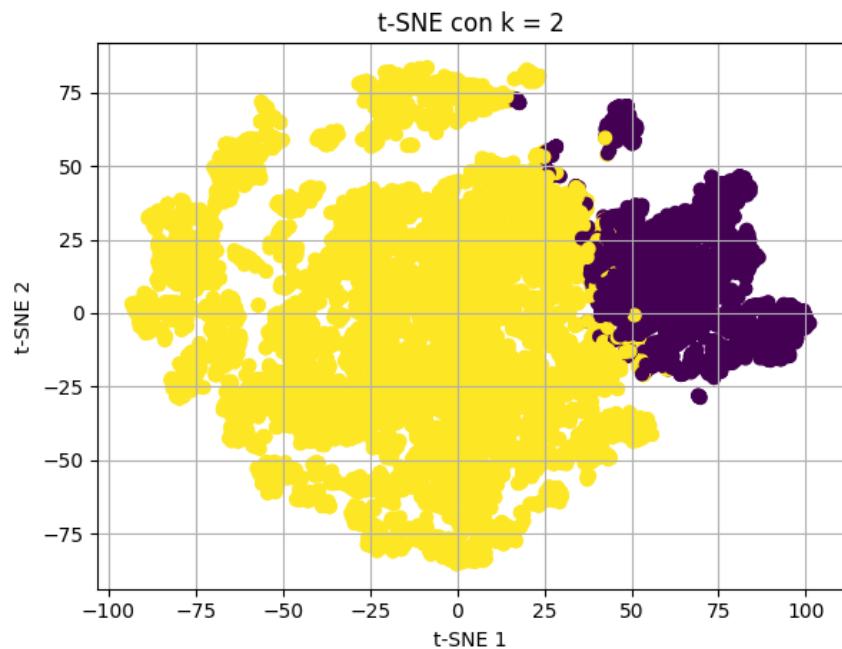
# 4. Probar diferentes valores de k
k_values = [2, 4, 5]
fig, axes = plt.subplots(1, len(k_values), figsize=(18, 5))

for i, k in enumerate(k_values):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
    labels = kmeans.fit_predict(X_scaled)

    axes[i].scatter(X_tsne[:, 0], X_tsne[:, 1], c=labels, cmap='viridis', s=40)
    axes[i].set_title(f't-SNE con k = {k}')
    axes[i].set_xlabel("t-SNE 1")
    axes[i].set_ylabel("t-SNE 2")
    axes[i].grid(True)

plt.suptitle("Comparación visual de clústers con t-SNE (k=2, 4, 5)", fontsize=16)
plt.tight_layout()
plt.subplots_adjust(top=0.88)
```

Comparación visual de clústers con t-SNE (k=2, 4, 5)



¿Qué muestra cada gráfico?

Cada panel usa la misma proyección 2D generada por t-SNE (basada en las variables originales escaladas), y colorea los puntos según los clústers asignados por **K-Means con diferentes valores de k** :

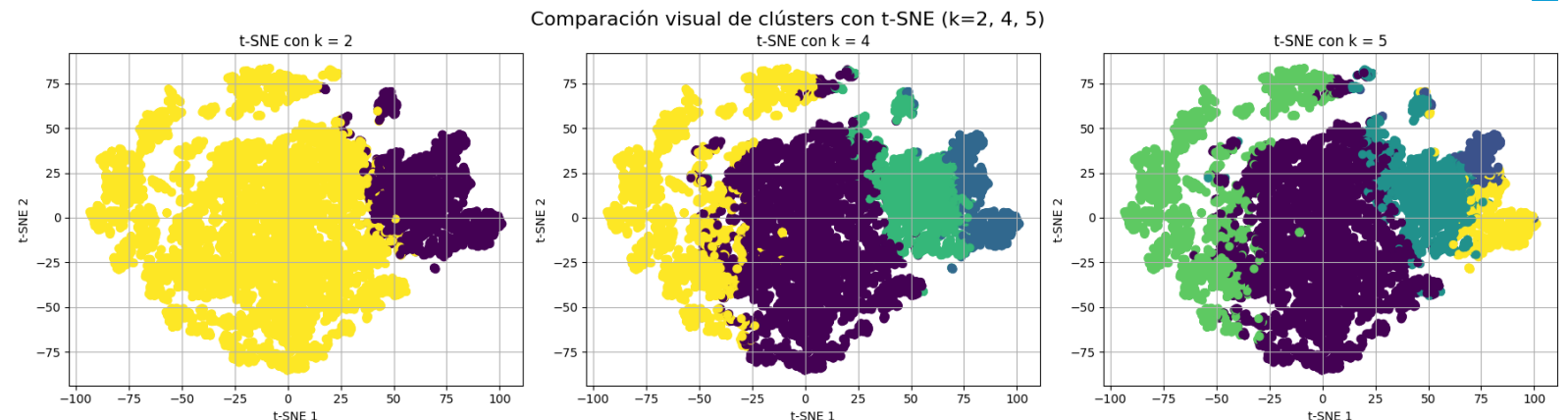
Izquierda: K-Means con $k = 2$

Centro: K-Means con $k = 4$

Derecha: K-Means con $k = 5$

Análisis visual

- ♦ **k = 2** Se observa **una división clara** entre dos regiones.
 - Los clústers están **relativamente bien separados**, aunque uno es mucho más grande.
 - Confirma lo que decía el **Silhouette Score** y el **Índice de Dunn**: buena calidad de separación con solo 2 clústers.
- ♦ **k = 4** Empiezan a aparecer **subgrupos diferenciados**.
 - La separación no es perfecta, pero hay **estructuras emergentes**, especialmente hacia la derecha.
 - Este valor fue sugerido por el **Método del Codo**, y se nota que hay una mejora de detalle sin demasiada confusión.
- ♦ **k = 5** El clúster central (color oscuro) **absorbe gran parte de los puntos**, lo que puede indicar:
 - Solapamiento entre grupos.
 - Un clúster muy dominante.
- Algunos nuevos colores aparecen, pero **la ganancia de detalle es marginal respecto a k = 4**.



k	Ventaja Principal	Desventaja	¿Cuándo usarlo?
2	Mayor separación, agrupamiento claro	Muy general, pierde matices	Para segmentación binaria (ej. bajo vs alto riesgo)
4	Buen detalle, sin mucho solapamiento	Un poco menos claro que $k = 2$	Ideal si quieres interpretabilidad + segmentación útil
5	Un poco más de granularidad	Clústeres menos definidos	Solo si realmente hay necesidad de más categorías

2. Métricas de Evaluación Internas

Las métricas internas nos dicen cómo de "buenos" son los clusters en función de las características de los propios datos. Algunas de las métricas más comunes son:

Índice de Silueta (Silhouette Score)

El **índice de silueta** mide lo **bien agrupados** están los puntos dentro de un mismo cluster y lo **separados** que están de otros clusters. Tiene un valor entre **-1 y 1**:

- **Valor cercano a 1**: Indica que los puntos están bien agrupados y muy separados de otros clusters.
- **Valor cercano a 0**: Indica que los puntos están en los límites entre dos clusters y podrían haber sido asignados incorrectamente.
- **Valor cercano a -1**: Indica que los puntos están mal asignados, ya que están más cerca de un cluster diferente.

¿Qué es el Silhouette Score?

El **Coeficiente de Silueta** mide qué tan bien separados están los clústers entre sí y qué tan compactos son internamente. Sus valores van de -1 a 1:

- **Cerca de 1:** los clústers están bien separados y compactos → excelente agrupamiento.
- **Cerca de 0:** los clústers se traslapan → agrupamiento débil.
- **Negativo:** mala asignación, los puntos podrían estar mejor en otro clúster.

¿Cómo interpretar la gráfica?

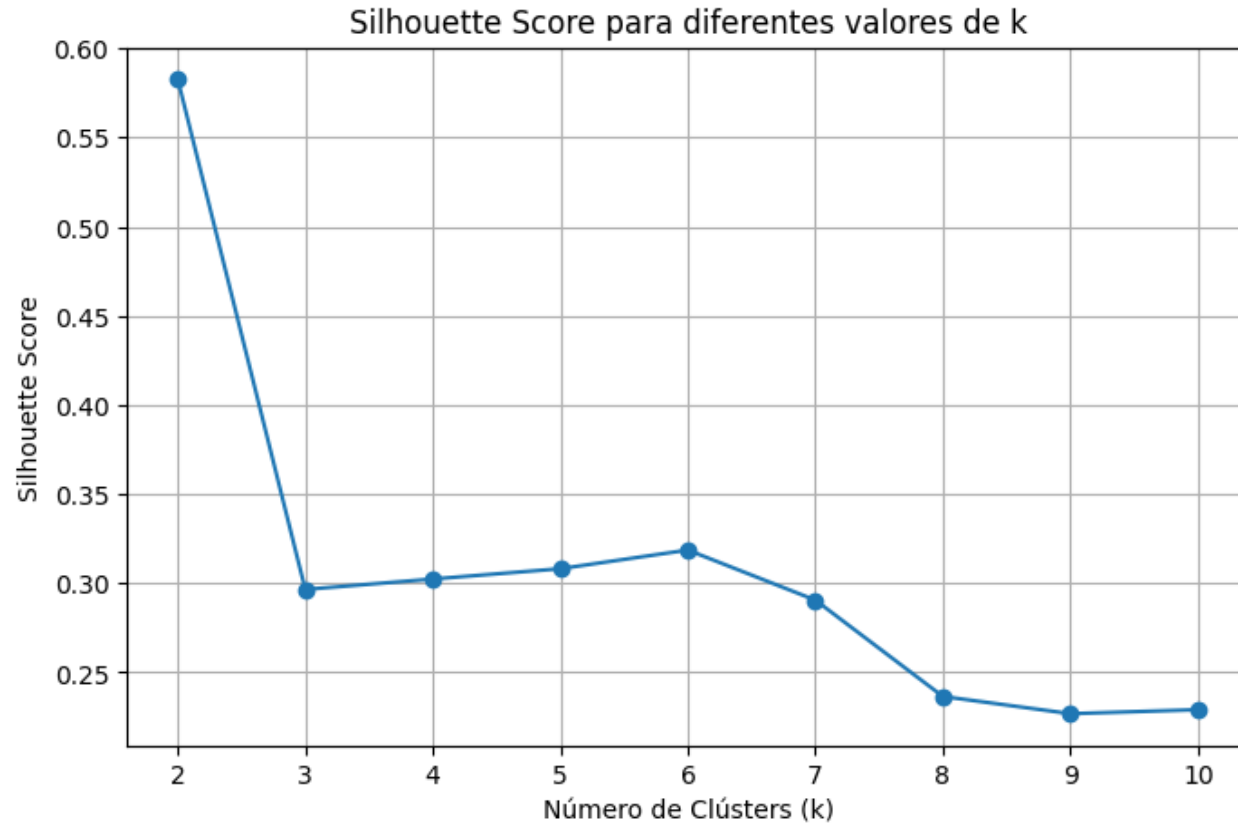
- El **pico más alto** del gráfico indica el k con mejor cohesión y separación.
- Compara con el valor sugerido por el Método del Codo ($k = 4$ o 5).
- Si el *score* cae mucho al aumentar k , significa que estás creando clústers poco significativos.

```
from sklearn.metrics import silhouette_score

# Evaluamos para k entre 2 y 10 (silhouette no tiene sentido con k=1)
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

# Graficar
plt.figure(figsize=(8,5))
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.title("Silhouette Score para diferentes valores de k")
plt.xlabel("Número de Clústers (k)")
plt.ylabel("Silhouette Score")
plt.grid(True)
plt.show()
```

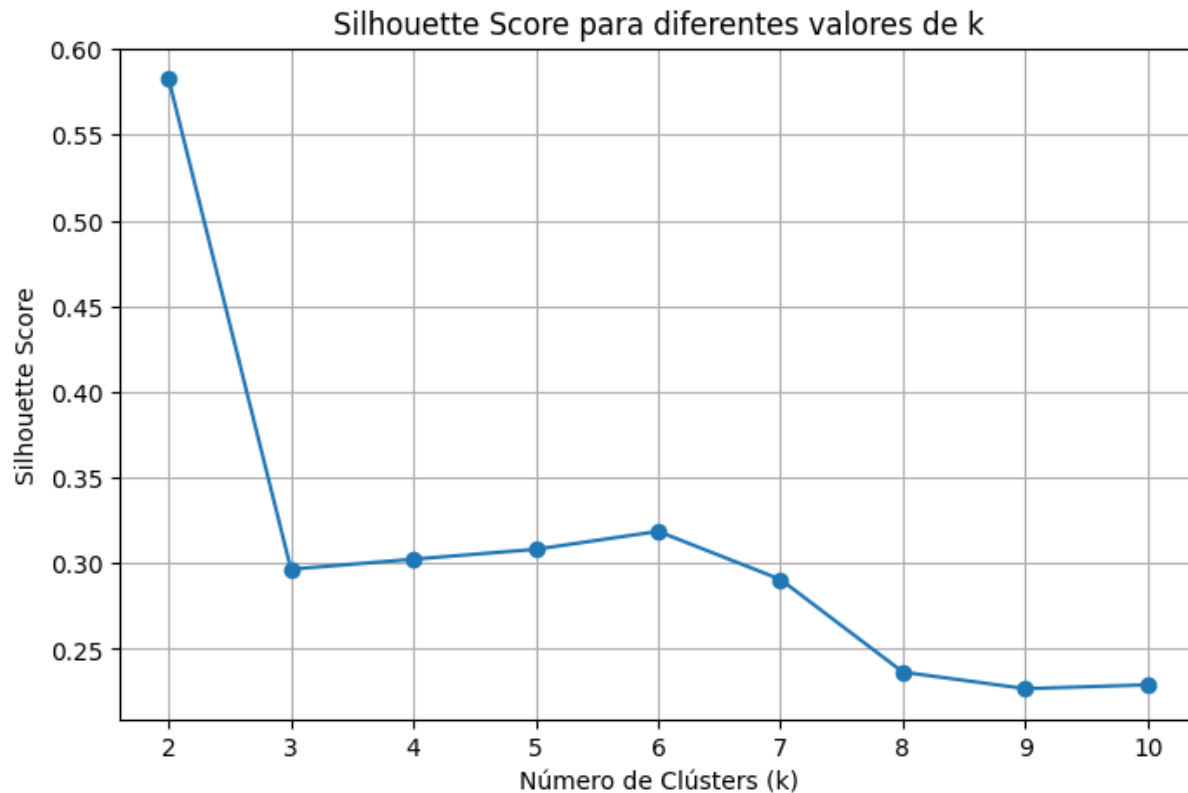


- **¿Qué muestra tu gráfico?**

Eje X: Número de clústers (k) probados, de 2 a 10.

Eje Y: Valor del **Silhouette Score**, que mide la calidad del agrupamiento.

▴ Valores más altos indican clústers **más separados y compactos**.



Observaciones clave

- **k = 2** tiene un **score muy alto (~0.58)**
→ indica **excelente separación** entre los 2 grupos.
- A partir de **k = 3 en adelante**, el Silhouette Score cae bruscamente y luego se mantiene entre **0.23 y 0.32**, lo cual sugiere:
 - **Menor cohesión y separación** entre clústers.
 - Posible solapamiento o clústers poco significativos.

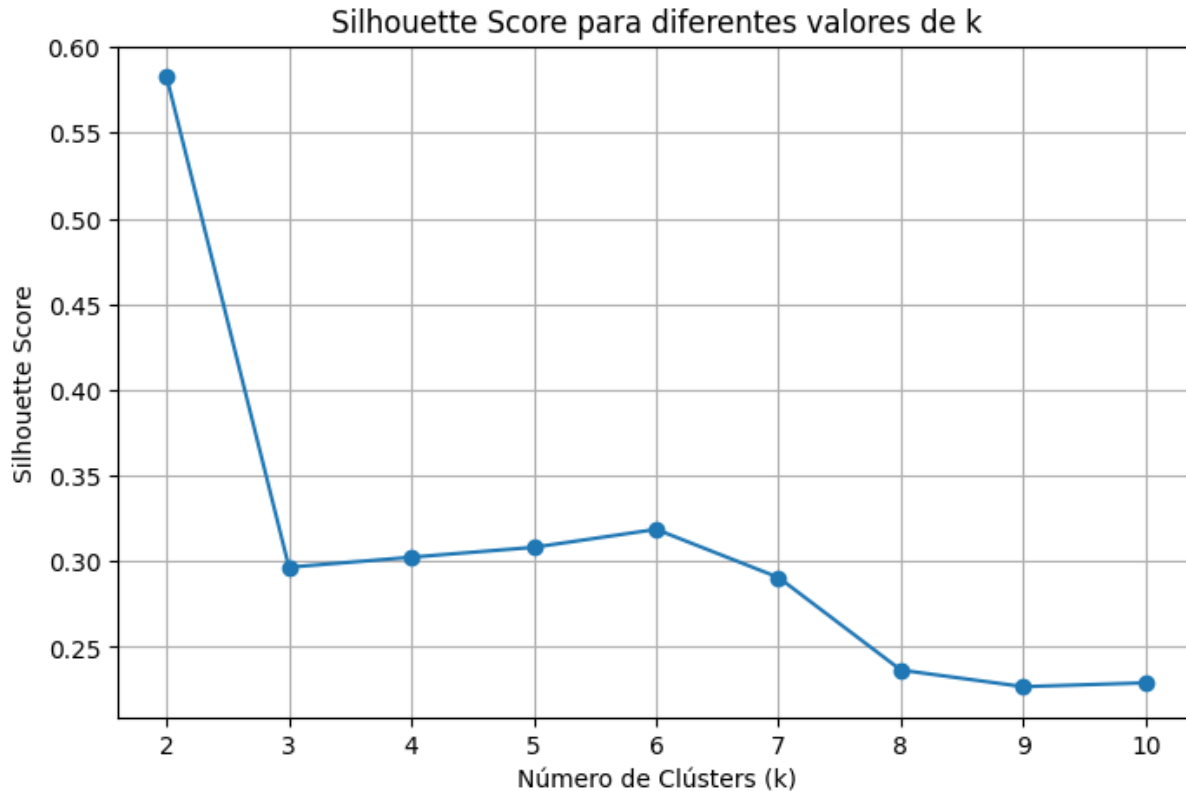
¿Qué significa esto?

Opción 1: $k = 2$

- Es el mejor valor según **Silhouette Score**.
- Tendrás **solo dos clústers**, pero bien definidos.
- Bueno si te interesa una agrupación muy general (ej. productos baratos vs costosos).

Opción 2: $k = 4$ o 5

- Aunque el score baja, sigue siendo aceptable (~ 0.30).
- Según el **método del codo**, estos valores daban mejor equilibrio entre simplicidad y detalle.
- Útil si quieres segmentaciones **más granulares** (ej. gama baja, media-baja, media-alta, alta).



-
- **Recomendación basada en ambos métodos**
 -  **Usa $k = 4$ si deseas un balance entre calidad y segmentación útil.**

k	Método del Codo	Silhouette Score	Conclusión
2	 Muy simple	 Muy alto	Demasiado general
4	 Punto de codo	 Medio-alto	Balanceado
5	 Punto de codo	 Medio	Detalle adicional

2. Métricas de Evaluación Internas

Las métricas internas nos dicen cómo de "buenos" son los clusters en función de las características de los propios datos. Algunas de las métricas más comunes es:

Índice de Dunn

- La **distancia mínima entre clústers diferentes** (es decir, qué tan separados están los clústers).
- Contra la **distancia máxima dentro de un mismo clúster** (qué tan dispersos son internamente).

Interpretación:

- **Mayor valor = mejor calidad** de agrupamiento.
- Altos valores significan:
 - Clústers bien separados.
 - Baja dispersión interna.



```

# 4. Definir función para calcular el Índice de Dunn
def dunn_index(X, labels):
    distances = pairwise_distances(X)
    unique_clusters = np.unique(labels)
    min_intercluster = np.inf
    max_intracluster = 0

    for i in unique_clusters:
        indices_i = np.where(labels == i)[0]
        if len(indices_i) > 1:
            intra_dists = distances[np.ix_(indices_i, indices_i)]
            max_intra = np.max(intra_dists)
            max_intracluster = max(max_intracluster, max_intra)
    for j in unique_clusters:
        if i != j:
            indices_j = np.where(labels == j)[0]
            inter_dists = distances[np.ix_(indices_i, indices_j)]
            min_inter = np.min(inter_dists)
            min_intercluster = min(min_intercluster, min_inter)

    if max_intracluster == 0:
        return 0
    return min_intercluster / max_intracluster

# 5. Calcular el índice de Dunn para k de 2 a 10
dunn_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
    labels = kmeans.fit_predict(X_scaled)
    score = dunn_index(X_scaled, labels)
    dunn_scores.append(score)

```

El código calcula el Índice de Dunn comparando qué tan dispersos son los puntos dentro de cada clúster (intra-cluster) y qué tan separados están entre sí (inter-cluster). Primero, se calcula una matriz de distancias entre todos los puntos. Luego, para cada clúster, se obtiene la **mayor distancia interna** (para medir su compacidad) y la **menor distancia hacia otro clúster** (para medir separación).

Finalmente, se divide la menor distancia entre clústers por la mayor distancia dentro de un clúster.

El resultado indica la calidad del agrupamiento: valores más altos significan clústers más definidos.

GRAFICA

En el eje:

- **X:** Número de clústers (k), de 2 a 10.
- **Y:** Valor del **Dunn Index** → mide la separación entre clústers comparada con la dispersión interna.

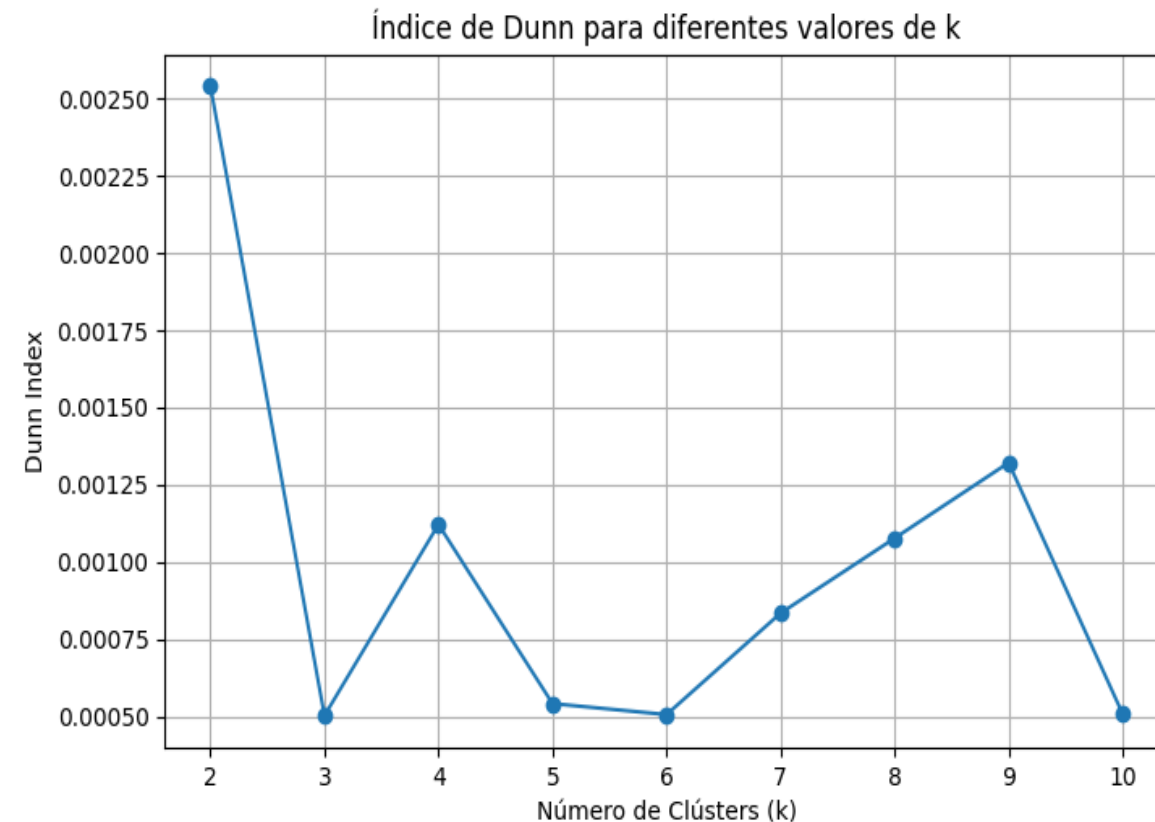
Observaciones clave del gráfico

- El **valor más alto** es en **$k = 2$** (~ 0.0025).
- El resto de valores cae significativamente.
- **$k = 4$ y $k = 9$** tienen valores ligeramente mejores que sus vecinos, pero **muy por debajo de $k = 2$** .

¿Qué significa?

- El **Índice de Dunn** sugiere que **$k = 2$** es la opción con **mejor separación entre clústers y baja dispersión interna**.
- Los valores para **$k > 2$** son muy bajos, lo que indica:
 - Clústers que se **solapan**.
 - Mayor **dispersión** dentro de algunos grupos.

```
# 6. Graficar resultados
plt.figure(figsize=(8,5))
plt.plot(range(2, 11), dunn_scores, marker='o')
plt.title("Índice de Dunn para diferentes valores de k")
plt.xlabel("Número de Clústers (k)")
plt.ylabel("Dunn Index")
plt.grid(True)
plt.show()
```



Comparación con los otros métodos

Interpretación

- Este dataset probablemente **forma 2 grupos bien definidos**, y más allá de eso, los clústers comienzan a solaparse. Esto puede ocurrir si los productos se agrupan de forma **binaria (por ejemplo: productos caros vs baratos)** y no en segmentos intermedios claramente diferenciables.

Método	Mejor k sugerido
Método del Codo	4 o 5
Silhouette Score	2 (clara ventaja)
Índice de Dunn	2 (máximo claro)

3. Validación del Número de Clusters: El Método del Codo

El **Método del Codo** es una técnica simple para determinar el número óptimo de clusters (k) en un algoritmo de clustering como K-Means. El algoritmo calcula la **inercia** (suma de las distancias cuadradas de los puntos a sus centroides) para diferentes valores de k y elige el número de clusters donde la disminución de la inercia comienza a estabilizarse.

¿Cómo se hace?

Elige varios valores de k (por ejemplo, de 1 a 10), ejecuta K-Means para cada valor de k y grafica la inercia. El punto donde la curva se aplanas es el número ideal de clusters.

```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.name))  
mirror_ob.select = 0  
= bpy.context.selected_objects  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES ----  
  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

¿Qué es el Método del Codo?

El Método del Codo (Elbow Method) busca encontrar el número de clústers k en el que agregar más clústers no mejora significativamente la calidad del modelo. Se basa en:

- Calcular la **Suma de los errores cuadráticos dentro del clúster (WCSS)** para diferentes valores de k .
- Graficar k frente al WCSS.
- Buscar el punto donde la curva se “dobla” como un codo: ese valor es el k ideal.



¿Qué es WCSS?

- Es la *Within-Cluster Sum of Squares*: mide cuán compactos son los clústers. Cuanto menor sea, mejor (los puntos están más cerca del centro del clúster).

¿Cómo interpretar?

- Observa el punto donde la reducción de WCSS se vuelve menos pronunciada.
 - Ese punto de “codo” sugiere un buen valor de k .
 - A veces no hay un codo muy evidente; en ese caso, puedes complementar con el **Coeficiente de Silueta** o probar diferentes k .

```
# 4. Aplicar Método del Codo
```

```
wcss = []
```

```
k_values = range(1, 11)
```

```
for k in k_values:
```

```
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
```

```
    kmeans.fit(X_scaled)
```

```
    wcss.append(kmeans.inertia_) # inertia_ es el WCSS
```

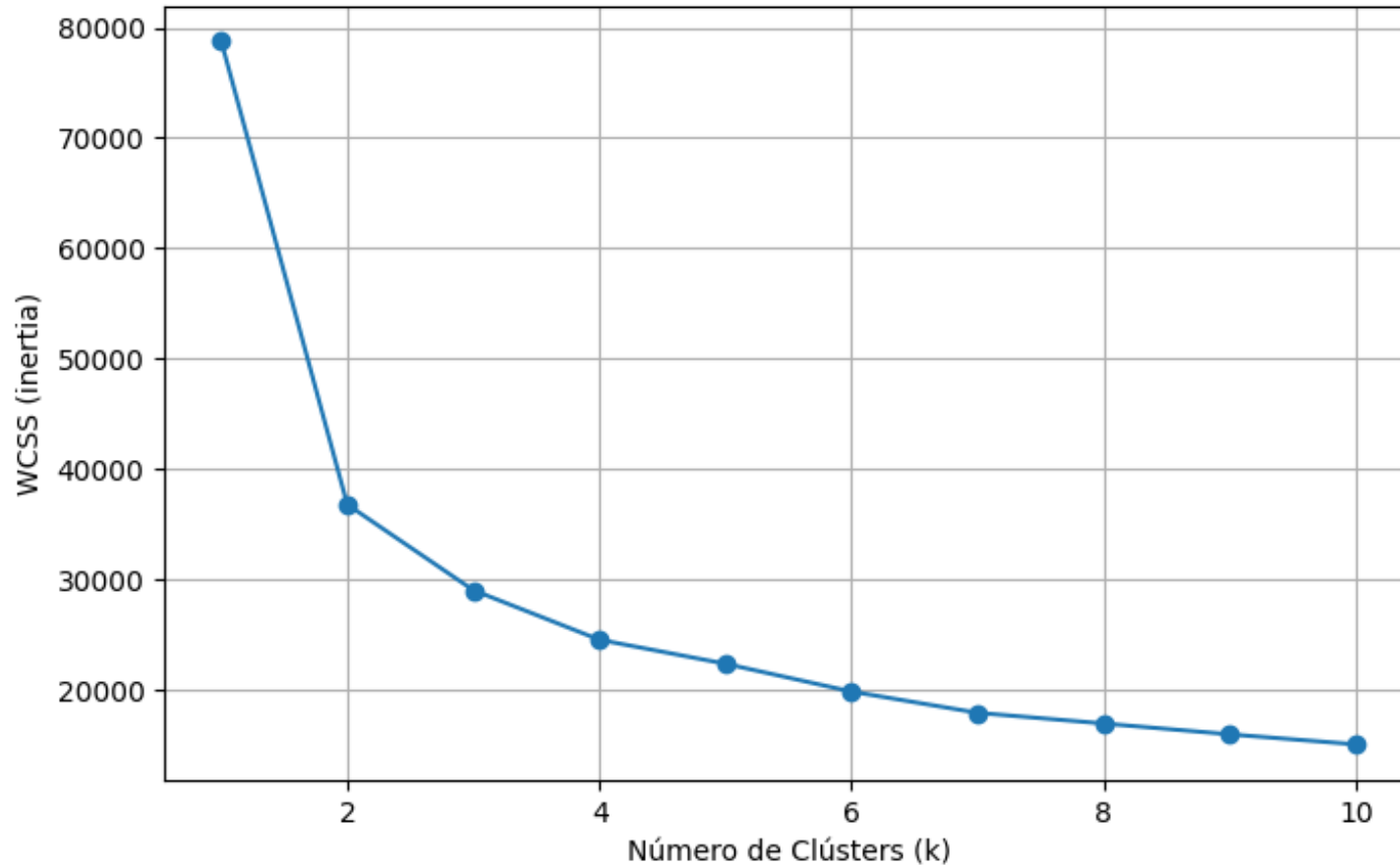
-
- `k_values = range(1, 11)`: vamos a probar desde 1 hasta 10 clústers.
 - `kmeans.inertia_`: es el **WCSS** que calcula automáticamente Scikit-learn.
 - Guardamos cada valor de WCSS para graficarlo luego.

```
# 5. Graficar
plt.figure(figsize=(8,5))
plt.plot(k_values, wcss, marker='o')
plt.title("Método del Codo para determinar k")
plt.xlabel("Número de Clústers (k)")
plt.ylabel("WCSS (inertia)")
plt.grid(True)
plt.show()
```

- Aquí se dibuja la curva que muestra cómo cambia el WCSS al aumentar k .
- Buscas visualmente el punto donde deja de bajar bruscamente = **el codo**.

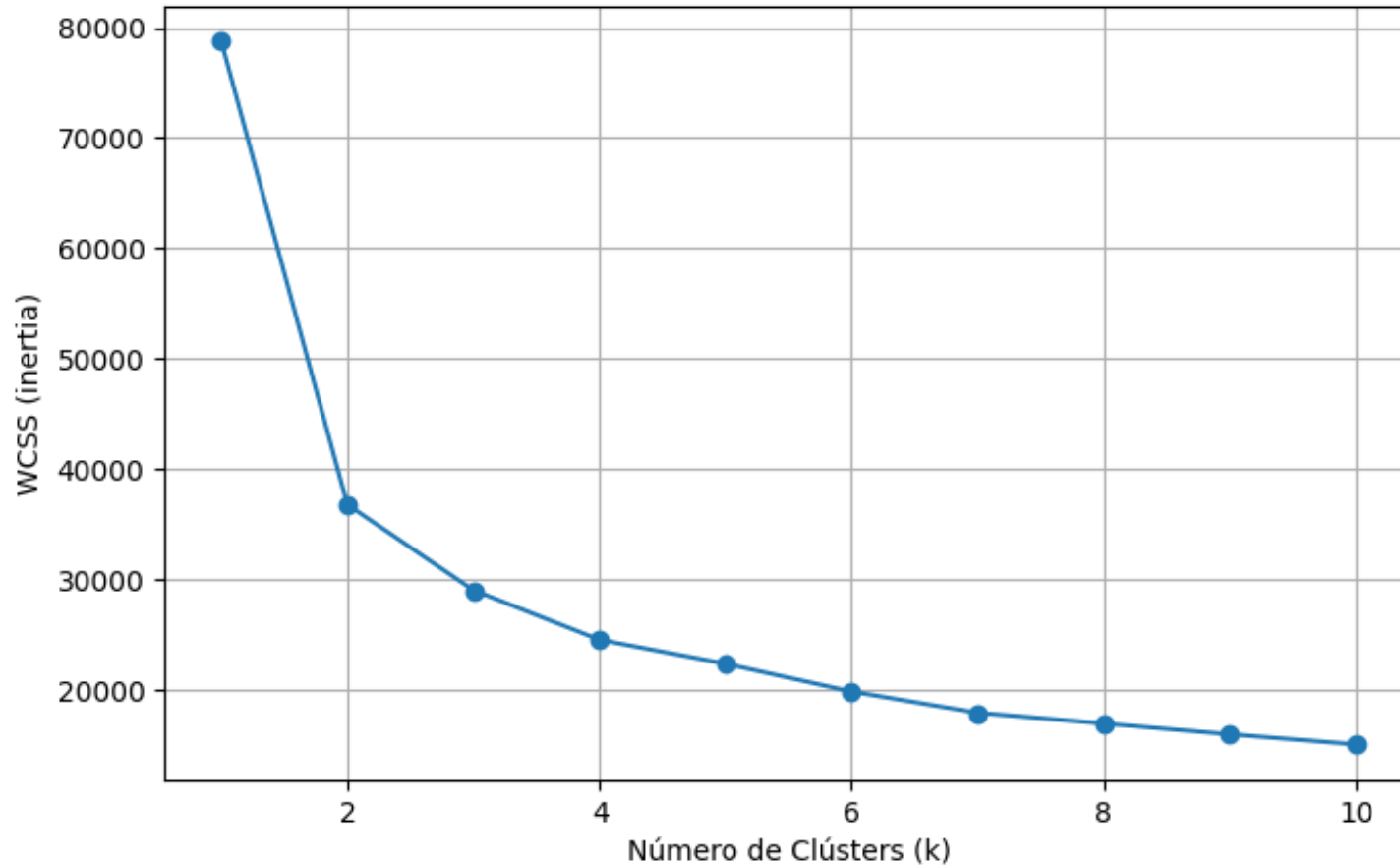


Método del Codo para determinar k



- **¿Qué muestra tu gráfico del Método del Codo?**
- En el eje:
- **X (horizontal):** el número de clústers (k) probados, de 1 a 10.
- **Y (vertical):** el valor de **WCSS (inercia)**, es decir, la suma de errores dentro de los clústers. Queremos que sea lo más bajo posible, pero sin crear demasiados clústers innecesarios.

Método del Codo para determinar k



¿Dónde está el "codo"?

Al observar la curva:

- De **k = 1 a k = 2**, hay una gran caída en WCSS: se mejora mucho el agrupamiento.
- De **k = 2 a k = 4**, la mejora sigue siendo visible.
- A partir de **k = 4 o 5**, las mejoras son mucho menores y la curva se aplan.

El codo parece estar en k = 4 o quizás k = 5.

Ese punto marca el equilibrio entre un modelo simple y una buena separación de los datos.



¿Qué significa esto para el análisis?

- Si eliges $k = 4$, obtendrás 4 grupos de productos con diferencias claras en precios/descuentos/costos.
- Si eliges $k = 5$, habrá un poco más de granularidad, pero ya con menor ganancia de precisión.
- En la práctica, puedes probar ambos y comparar la interpretación de los clústers.

¿Qué sigue?

Si quieres afinar la elección de k , puedes usar:

- **Coeficiente de silueta** (Silhouette Score), que mide cuán bien separado está cada clúster.
- **Visualización de los clústers** con PCA (reducción a 2D) para entender si tienen sentido.

Resumen de Técnicas para Evaluar Clustering:

Técnica	Descripción	Cómo evaluar
Visualización	Reducción de dimensionalidad (PCA, t-SNE)	Los clusters deben ser visualmente coherentes.
Índice de Silueta	Evalúa la separación entre clusters	Un valor cercano a 1 indica buen clustering.
Índice de Dunn	Mide la separación entre clusters y la compactación	Un valor alto indica una buena separación.
Método del Codo	Busca el número óptimo de clusters (inercia)	El "codo" indica el número ideal de clusters.

Conclusión:

- Para saber si el clustering tiene sentido, se puede usar una combinación de **métricas** como el **Silhouette Score**, el **Método del Codo**, y la **visualización** de los resultados. Si los clusters son coherentes tanto **visual como numéricamente**, entonces el clustering es probablemente adecuado. Si no, puede que sea necesario ajustar el número de clusters o probar con otras características.

Métrica	¿Qué mide?	¿Cuándo tiende a favorecer k pequeño?	¿Cuándo es útil?	Ejemplo de aplicación ideal
Índice de Dunn	Separación mínima entre clústers / dispersión máxima interna	Cuando hay clústers muy separados	Cuando se busca separación clara y compacta	Diagnóstico binario (ej. sano vs enfermo)
Silhouette Score	Cohesión y separación relativa de cada punto	A veces (si hay un k con clústers bien definidos)	Cuando se busca un equilibrio por punto	Agrupamiento de clientes por comportamiento
Método del Codo	Reducción de WCSS (error cuadrático interno)	Tiende a crecer con $k \rightarrow$ buscar el “codo”	Para encontrar el punto de saturación óptimo	Agrupamiento operativo o logístico

C2 TALLER2

Con base en sector, temática
y datasets seleccionados:
*aplicar KMEANS

FECHA: Martes 23 cada
grupo presenta resultados y
hallazgos.